

A PROJECT REPORT ON

**LEADS AND REPORT ANALYTICS USING MACHINE
LEARNING ON INSURANCE SALES DATA**

SUBMITTED TO THE VISHWAKARMA INSTITUTE OF INFORMATION TECHNOLOGY,
PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

**BACHELOR OF TECHNOLOGY (COMPUTER SCIENCE &
ENGINEERING (AIML))**

SUBMITTED BY

STUDENT NAME

Gauri Mane
Sarthak Hambir
Raghav Bhise
Joel Cherian

PRN

22311783
22311784
22311792
22311799



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(AIML)**

**BRACT'S
VISHWAKARMA INSTITUTE OF INFORMATION TECHNOLOGY**

SURVEY NO. 3/4, KONDHWA (BUDRUK), PUNE – 411048, MAHARASHTRA (INDIA).

**BRACT'S
VISHWAKARMA INSTITUTE OF INFORMATION TECHNOLOGY, PUNE**

2025 -2026



CERTIFICATE

This is to certify that the project report entitles

**“ LEADS AND REPORT ANALYTICS USING MACHINE LEARNING ON INSURANCE
SALES DATA”**

Gauri Mane, Sarthak Hambir, Raghav Bhise and Joel Cherian are bonafide student of this institute and the work has been carried out by her under the supervision of **Mrs. Gitanjali Shinde** and it is approved for the partial fulfillment of the requirement of VISHWAKARMA INSTITUTE OF INFORMATION TECHNOLOGY, for the Third Year Project of **BACHELOR OF TECHNOLOGY** (COMPUTER SCIENCE & ENGINEERING (AIML)).

(Mrs. Gitanjali Shinde)
Guide
Department of CSE-AIML

(Dr. Premanand Ghadekar)
Head,
Department of CSE-AIML

(Dr. Vivek Deshpande)
Director,
BRACT's Vishwakarma
Institute of Information
Technology, Pune-48

Place : Pune
Date :

ACKNOWLEDGEMENT

We would like to express our sincere and heartfelt gratitude to our internal guide, **Mrs. Gitanjali Shinde**, and all the faculty members of the **Department of Computer Science and Engineering (AIML)** for their invaluable guidance, continuous encouragement, and unwavering support throughout the duration of this project. Their insightful suggestions, timely feedback, and dedicated involvement have played a crucial role in helping us understand the technical and conceptual aspects more clearly. We are truly grateful for the knowledge and motivation they have provided at every stage of this work.

We are deeply thankful to **SimpleSight Solutions Pvt. Ltd.** for offering meaningful insights into real-world industry requirements and expectations. Their professional input, practical exposure, and willingness to share expertise significantly enhanced the applied dimension of this project. The opportunity to learn about industry practices not only strengthened our understanding but also helped in shaping the overall direction and relevance of this work.

Furthermore, we would like to acknowledge the constant support and cooperation of our friends, whose helpful discussions, shared resources, and encouragement kept us motivated throughout this journey. Their presence made challenging phases much easier to navigate. Lastly, we extend our heartfelt appreciation to our family for their endless patience, unconditional support, and belief in our abilities. Their encouragement, understanding, and emotional strength have been the backbone of our efforts, enabling us to complete this project with commitment and confidence.

Gauri Mane (391056)

Sarthak Hambir (391057)

Raghav Bhise (391061)

Joel Cherian (391064)

(B. Tech Computer Science and Engineering (AI & ML))

ABSTRACT

The insurance broking industry often operates with fragmented data systems and manual workflows, making it difficult to efficiently manage leads, track policies, and retain customers. Sales agents lack a clear, real-time understanding of which leads are most valuable, while brokers struggle to identify customers who may not renew their policies. This results in wasted effort, missed opportunities, and reduced revenue. The project addresses these challenges by proposing a centralized, data-driven system that integrates customer information, lead management, policy tracking, payments, and commissions into a single platform, providing a structured and unified view of business operations.

To overcome these limitations, the system leverages machine learning techniques to enhance decision-making. A lead conversion model is used to assign a probability score indicating the likelihood of a lead converting into an active policy, allowing agents to prioritize high-value prospects. Additionally, a predictive churn model analyzes factors such as payment behavior, policy tenure, and customer attributes to identify policies at risk of non-renewal. The backend is implemented using a PostgreSQL database (via Supabase) to ensure efficient data storage and retrieval, while the frontend is developed using FlutterFlow to present intuitive dashboards, list views, and analytics for users.

The proposed solution enables brokers to make informed, proactive decisions by transforming raw data into actionable insights. By prioritizing high-conversion leads and identifying at-risk customers in advance, the system improves operational efficiency, enhances customer retention, and supports revenue growth. Overall, this project demonstrates how the integration of structured data management with machine learning can significantly optimize insurance broking processes and deliver measurable business value.

TABLE OF CONTENTS

LIST OF ABBREVIATIONS	i
LIST OF FIGURES	ii
LIST OF TABLES	iii

CHAPTER	TITLE	PAGE NO.
Sr. No.	Title of Chapter	Page No.
01	Introduction	8
1.1	Overview	8
1.2	Motivation	9
1.3	Problem Definition and Objectives	10
1.4	Project Scope & Limitations	12
1.5	Methodologies of Problem solving	13
02	Literature Survey	15
2.1	Related Work	15
2.2	Gap Analysis	17
03	Software Requirements Specification	18
3.1	Assumptions and Dependencies	18
3.2	Functional Requirements	19
3.2.1	Customer and Lead Management	20
3.2.2	Policy Management	20
3.2.3	Payment and Commission Management	20
3.2.4	Predictive Churn Analysis	21
3.2.5	Lead Conversion Scoring	21
3.2.6	Dashboard and Reporting	22
3.2.7	Document and Endorsement Management	22
3.2.8	User and Role Management	22
3.3	Nonfunctional Requirements	22
3.3.1	Performance Requirements	23
3.3.2	Safety Requirements	23
3.3.3	Security Requirements	23
3.3.4	Software Quality Attributes	24
3.4	System Requirements	24
3.4.1	Database Requirements	25
3.4.2	Software Requirements (Platform Choice)	25
3.4.3	Hardware Requirements	25
3.5	Analysis Models: SDLC Model to be applied	26
04	System Design	27
4.1	System Architecture	27
4.2	Mathematical Model	30
4.3	Data Flow Diagrams	33

4.4	Entity Relationship Diagrams	35
4.5	UML Diagrams	35
05	Project Plan	38
5.1	Project Estimate	38
5.2	Project Schedule - Timeline Chart	39
06	Project Implementation	41
6.1	Overview of Project Modules	41
6.2	Tools and Technologies Used	42
6.3	Algorithm Details	43
6.3.1	Algorithm 1 - Predictive Renewal Churn Model	42
6.3.2	Algorithm 2 - Lead Conversion Scoring Model	43
6.3.3	Additional Notes	43
07	Results	43
7.1	Outcomes	43
7.2	Screen Shots	44
08	Conclusions and Future Work	49
8.1	Applications	50
	References	52
	APA format.	
9	Appendix	53
	A: Plagiarism Report of project report.	53
	B: Project offer letter	54
	C: Project completion letter	55

INTRODUCTION

1.1 OVERVIEW

The project titled “**Leads and Report Analytics using Machine Learning on Insurance Sales Data**” is aimed at developing a comprehensive and intelligent system for managing insurance broking operations. In the current scenario, insurance brokers handle large volumes of data related to customers, leads, policies, payments, and commissions. However, this data is often stored in fragmented systems such as spreadsheets or manual records, making it difficult to access, analyze, and utilize effectively. This project addresses these challenges by introducing a centralized platform that integrates all key functionalities into a single structured database, enabling efficient data management and seamless workflow across different stages of the insurance lifecycle.

In addition to data organization, the system incorporates machine learning techniques to enhance decision-making capabilities. A lead conversion model is used to evaluate and rank leads based on their likelihood of converting into active policies, allowing sales agents to prioritize high-value opportunities. Similarly, a predictive churn model analyzes customer behavior and policy-related factors to identify clients who are at risk of not renewing their policies. These predictive insights transform the system from a simple data management tool into a decision-support system, helping brokers take proactive actions to improve sales performance and customer retention.

The frontend of the system is developed using FlutterFlow, providing a user-friendly web interface with dashboards, list views, and analytics for real-time monitoring. The backend is implemented using a PostgreSQL database through Supabase, ensuring reliable data storage and efficient query handling. Synthetic data is generated to simulate real-world scenarios and support the training of machine learning models. Overall, the project demonstrates how the integration of structured database systems with machine learning can streamline insurance broking operations, reduce manual effort, and enable data-driven business strategies. (Fig. 1)

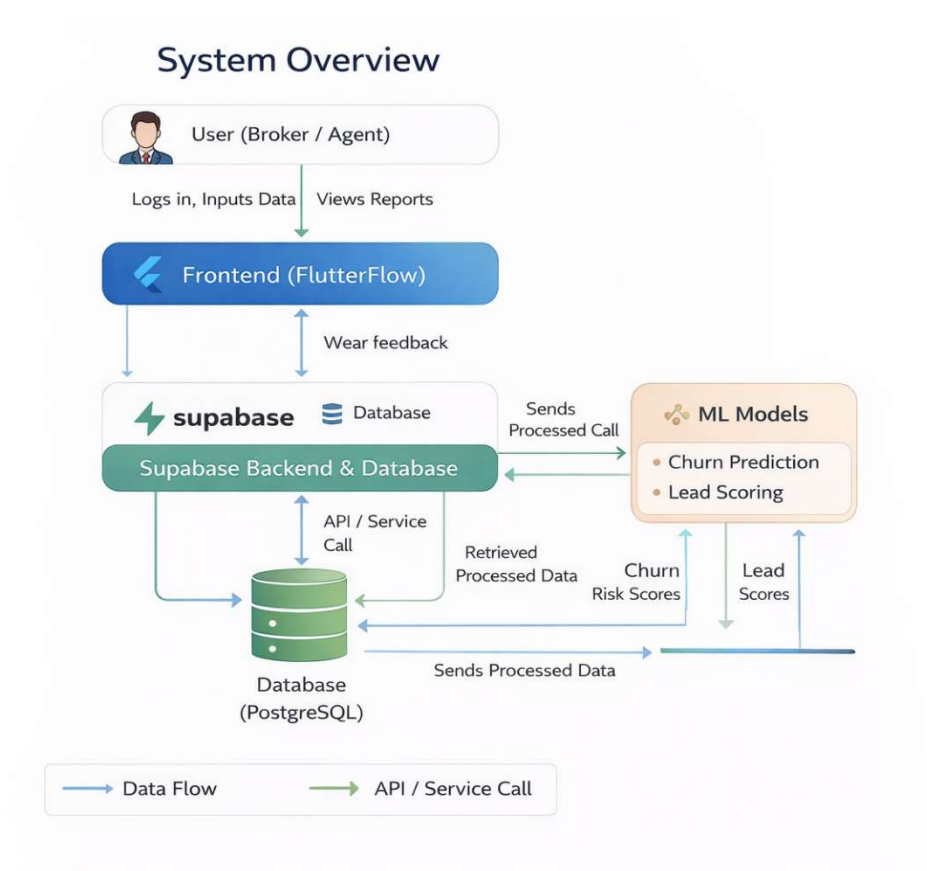


Fig.1 System Overview

1.2 MOTIVATION

The motivation for this project stems from the growing challenges faced by insurance broking companies in managing large volumes of data and making timely, informed decisions. In many organizations, data related to customers, leads, policies, and payments is maintained in fragmented formats such as spreadsheets or isolated systems. This lack of integration makes it difficult to obtain a clear, real-time view of business performance. As a result, sales agents often spend time on low-quality leads without knowing their actual potential, and important insights that could improve decision-making remain hidden within scattered data. This inefficiency directly impacts productivity and revenue generation.

Another major concern is the absence of predictive capabilities in traditional systems. Brokers often struggle to identify which customers are likely to renew their policies and which ones are at risk of discontinuation. Without such insights, retention strategies are reactive rather than proactive, leading to missed opportunities for customer engagement. Similarly, the inability to prioritize leads based on their likelihood of conversion results in inefficient use of time and resources. These

challenges highlight the need for a system that not only organizes data but also extracts meaningful patterns and predictions from it.

This project is motivated by the idea of transforming conventional insurance operations into a data-driven ecosystem. By integrating machine learning with a centralized database, the system aims to provide actionable insights such as lead conversion probability and churn risk. This enables brokers to focus on high-value opportunities, improve customer retention, and optimize overall business performance. The goal is to bridge the gap between data availability and intelligent decision-making, ultimately enhancing efficiency, accuracy, and competitiveness in the insurance broking domain. (Fig. 2)

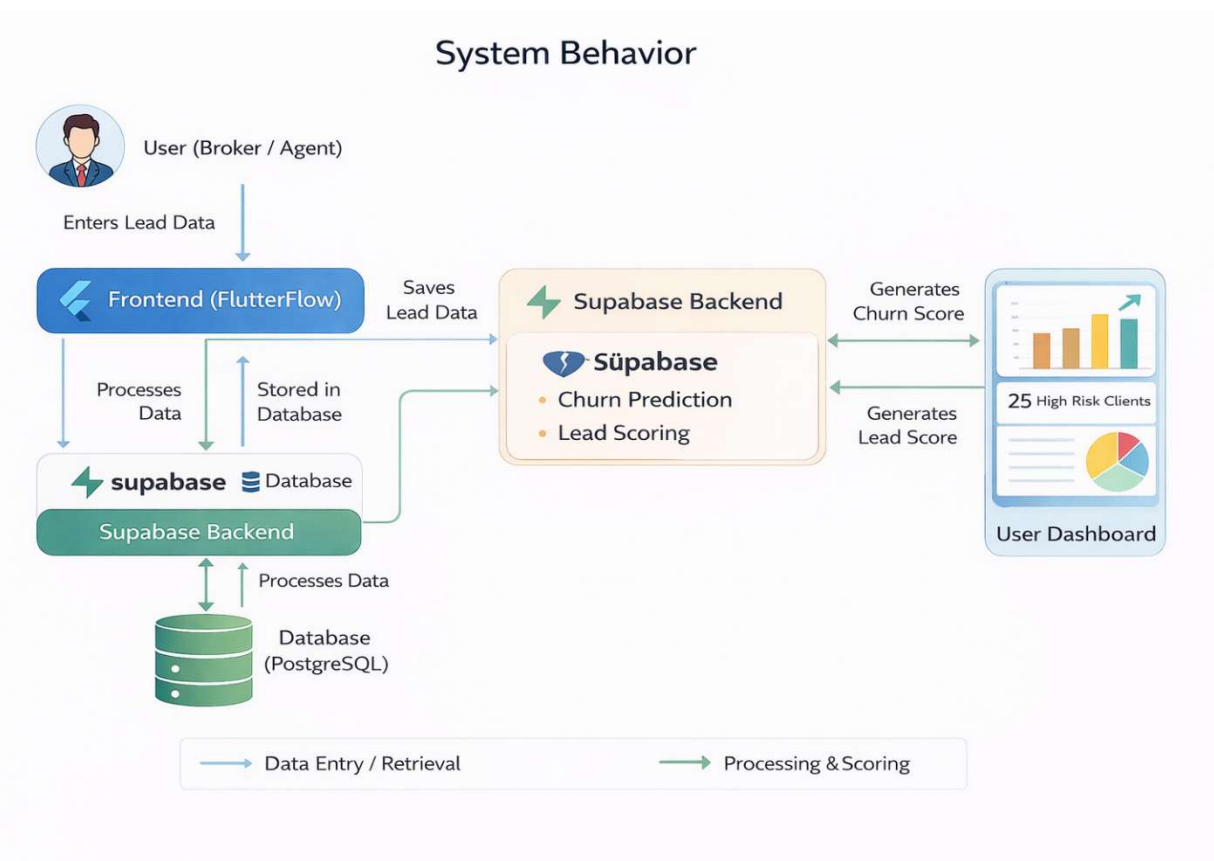


Fig. 2 System Behavior

1.3 PROBLEM DEFINITION AND OBJECTIVES

1.3.1 Problem Definition

Insurance broking companies operate in a highly data-intensive environment where information related to customers, leads, policies, payments, and commissions must be managed efficiently.

However, in many cases, this data is stored across multiple disconnected systems such as spreadsheets, legacy applications, or manual records. This fragmented approach leads to inconsistencies, duplication, and difficulty in retrieving accurate information when needed. As a result, brokers lack a unified view of their operations, making it challenging to track the complete lifecycle of a customer from lead generation to policy renewal. This inefficiency not only slows down business processes but also increases the chances of errors and missed opportunities.

A significant problem arises in decision-making due to the absence of analytical and predictive tools. Sales agents often do not have a clear understanding of which leads are more likely to convert into policies, leading to wasted effort on low-potential prospects. Similarly, brokers are unable to identify customers who are at risk of not renewing their policies, resulting in poor retention rates. Without predictive insights, business strategies remain reactive rather than proactive. Additionally, manual calculation and tracking of commissions further complicate operations, increasing the workload and reducing overall efficiency. These challenges highlight the need for a system that can centralize data, automate processes, and provide intelligent insights for better decision-making.

1.3.2 Objectives

The primary objective of this project is to design and develop a centralized, data-driven insurance broking system that integrates operational processes with machine learning-based analytics. The system aims to streamline data management, enhance decision-making, and improve overall business efficiency. By combining a structured database with predictive models, the project focuses on transforming traditional insurance workflows into an intelligent and automated platform that supports both sales optimization and customer retention.

The specific objectives of the project are as follows:

- To design a structured relational database for managing customers, leads, policies, payments, commissions, and endorsements efficiently.
- To develop a centralized platform that eliminates data fragmentation and ensures consistency and easy accessibility of information.
- To implement a lead conversion model that assigns probability scores, enabling sales agents to prioritize high-value leads.
- To develop a predictive churn model to identify customers who are likely to discontinue their policies and support proactive retention strategies.
- To create an intuitive user interface using FlutterFlow for effective data visualization, dashboards, and reporting.

- To automate key processes such as commission calculation and tracking, reducing manual effort and minimizing errors.
- To improve overall operational efficiency, decision-making, and revenue generation through data-driven insights.

1.4 PROJECT SCOPE AND LIMITATIONS

1.4.1 Scope

The scope of this project focuses on the design and development of a centralized insurance broking system that integrates operational management with analytical capabilities. The system covers the complete lifecycle of insurance processes, including customer onboarding, lead generation and tracking, policy issuance, premium payment recording, commission management, and document handling. It is specifically tailored for Life and Health insurance domains, ensuring that policy-specific details are handled appropriately within the database structure. By organizing all business-related data into a single platform, the system enables efficient data storage, retrieval, and management, reducing dependency on fragmented tools and manual processes.

In addition to operational functionalities, the project extends its scope to include machine learning-based analytics for improved decision-making. The system incorporates models for lead conversion scoring and predictive churn analysis, allowing brokers to identify high-value leads and at-risk customers. These insights are presented through a user-friendly web interface developed using FlutterFlow, which includes dashboards, list views, and analytical reports. The integration of Supabase (PostgreSQL) ensures reliable backend support, while synthetic data generation is used to simulate real-world scenarios for testing and model training. Overall, the project aims to bridge the gap between data management and intelligent analytics within the insurance broking ecosystem.

1.4.2 Limitations

Despite its comprehensive design, the project has certain limitations primarily due to its academic and prototype nature. One of the key limitations is the use of synthetic data instead of real-world datasets. While the generated data closely mimics real scenarios, it may not fully capture the complexity, variability, and unpredictability of actual insurance data. As a result, the performance of machine learning models may differ when applied to real-world environments. Additionally, the predictive models implemented in the system are relatively simplified and may not incorporate advanced techniques or large-scale training required for production-level deployment.

Another limitation is the absence of integration with external systems such as insurance provider APIs, payment gateways, or regulatory databases. The system operates as a standalone platform, which restricts its ability to fetch real-time data or perform live transactions. Furthermore, the frontend developed using FlutterFlow is optimized for rapid development and prototyping, which may limit customization and scalability for enterprise-level applications. Performance constraints may also arise when handling extremely large datasets or concurrent users. These limitations indicate that while the project successfully demonstrates the concept of a data-driven insurance system, further enhancements and real-world integrations would be required for full-scale deployment.

1.5 Methodologies of Problem Solving

The development of this project adopts a comprehensive and systematic methodology to effectively address the challenges associated with insurance broking operations, such as fragmented data management, inefficient lead prioritization, and lack of predictive insights. The approach begins with a detailed analysis of existing workflows within insurance systems to understand how data flows across different stages, from customer acquisition to policy renewal. Based on this understanding, a structured problem-solving framework is defined, integrating database design, data processing, and machine learning techniques. The goal is to transform raw and scattered data into meaningful, actionable insights that can support better decision-making and operational efficiency.

A key aspect of the methodology is the design of a robust relational database using PostgreSQL through Supabase. The database is carefully structured to maintain relationships between entities such as customers, leads, policies, payments, commissions, and endorsements. This ensures data consistency, eliminates redundancy, and enables efficient querying. To simulate real-world scenarios, synthetic data is generated using Python libraries, allowing the system to handle large datasets and test different use cases without relying on sensitive real data. This step plays a crucial role in preparing the dataset for machine learning applications, ensuring that the models are trained on diverse and realistic inputs.

Machine learning forms the core of the analytical component of the project. Supervised learning techniques are applied to develop models for lead conversion scoring and churn prediction. The lead conversion model evaluates various attributes such as customer profile, expected premium, and historical patterns to assign a probability score indicating the likelihood of conversion. Similarly,

the churn prediction model analyzes factors like payment behavior, policy tenure, and customer risk indicators to identify policies that are likely to lapse. These models are integrated into the system so that their outputs can be stored in the database and utilized directly within the application interface. The frontend development is carried out using FlutterFlow, which enables rapid creation of a responsive and user-friendly web application. The interface is designed to present complex data in a simplified manner through dashboards, list views, and filters. Features such as priority-based lead ranking and churn risk visualization allow users to focus on high-impact areas without needing deep technical knowledge. The integration between the frontend and backend ensures that data flows seamlessly, providing real-time insights to users.

The methodologies adopted in this project can be summarized as follows:

- **Requirement Analysis:** Understanding the core challenges in insurance broking, including data fragmentation, lack of centralized systems, and absence of predictive decision-making tools.
- **System Design and Database Modeling:** Designing a normalized relational database schema that defines clear relationships between entities and ensures data integrity and scalability.
- **Synthetic Data Generation:** Creating realistic datasets using Python to simulate customer behavior, transactions, and policy lifecycle events for testing and model training.
- **Machine Learning Implementation:** Applying supervised learning algorithms such as Logistic Regression and ensemble methods to develop predictive models for lead conversion and churn analysis.
- **Data Processing and Integration:** Combining operational data with machine learning outputs, storing scores and predictions in the database for seamless access and analysis.
- **Frontend Development:** Developing an interactive web interface using FlutterFlow to visualize data through dashboards, tables, and filters, enabling easy user interaction.
- **Testing and Validation:** Performing iterative testing to ensure accuracy of models, correctness of data flow, and reliability of system functionalities.
- **Iterative Improvement:** Continuously refining the system based on testing results, enhancing performance, usability, and scalability.

This structured and layered methodology ensures that the project effectively solves the identified problems while providing a scalable and adaptable solution. By integrating database systems,

machine learning, and user interface design, the approach not only addresses current challenges but also lays a strong foundation for future enhancements and real-world deployment.

LITERATURE SURVEY

2.1 RELATED WORK

The insurance industry has witnessed significant advancements with the adoption of digital platforms and data-driven technologies. Many leading insurance companies such as **LIC**, **HDFC Life**, **ICICI Prudential**, and **SBI Life** have developed their own digital ecosystems to manage customer data, policies, and transactions. These systems typically include customer portals, agent dashboards, and backend databases that streamline operations like policy issuance, premium payments, and document management. While these platforms improve operational efficiency and accessibility, they are largely focused on **transactional processing** rather than **predictive analytics**.

Most of these companies utilize internal systems that provide structured data storage and basic reporting features. For example, agents can view customer details, track policy status, and monitor payments through dashboards. However, these dashboards often rely on historical data visualization rather than forward-looking insights. The absence of integrated machine learning models limits the ability of these systems to predict customer behavior, such as identifying high-conversion leads or detecting customers likely to churn. As a result, decision-making remains largely manual and experience-driven, rather than being supported by intelligent automation.

In recent years, some insurance companies have started experimenting with data analytics and machine learning for specific use cases such as fraud detection and risk assessment. However, these implementations are usually confined to backend analytical teams and are not directly accessible to sales agents or brokers in real time. This creates a disconnect between data analysis and practical application, where valuable insights are not fully utilized in day-to-day operations. Additionally, many systems operate in silos, where CRM functionalities and analytical tools are not seamlessly integrated, leading to inefficiencies in workflow.

The proposed system in this project differs significantly by providing a **fully integrated, data-driven platform** that combines operational management with machine learning insights. Unlike traditional systems, it enables **real-time lead scoring** and **churn prediction**, allowing brokers to

make proactive decisions. The system is designed to present these insights directly within the user interface, ensuring that non-technical users can easily interpret and act upon them.

The following table provides a comparison between existing insurance company systems and the proposed solution:

Feature	Existing Insurance Systems (LIC, HDFC Life, ICICI, SBI Life)	Proposed System
Data Management	Structured but often fragmented across modules	Centralized relational database
Analytics	Mainly descriptive (reports, dashboards)	Predictive analytics using ML models
Lead Handling	Manual prioritization by agents	Automated lead scoring (conversion probability)
Customer Retention	Limited tools for renewal tracking	Churn prediction with risk scoring
Integration	CRM and analytics often separate systems	Unified platform (data + ML + UI)
Decision Making	Based on experience and past data	Data-driven and predictive insights
User Interface	Functional but less interactive	Interactive dashboards using FlutterFlow
Automation	Limited automation	Automated insights and prioritization

In conclusion, while existing insurance systems have made significant progress in digitizing operations, they still lack the integration of **real-time predictive analytics** within their workflows. The proposed system addresses this gap by combining structured data management with machine learning and intuitive visualization, thereby enabling a more **efficient, intelligent, and proactive approach** to insurance broking.

2.2 GAP ANALYSIS

Although current systems and research efforts have contributed to improving insurance operations, several critical gaps still exist that limit their effectiveness in real-world applications. One of the

primary issues is the lack of a centralized and unified system that integrates all aspects of insurance broking. In many organizations, data related to customers, leads, policies, and payments is distributed across multiple platforms or stored in spreadsheets. This fragmentation leads to inconsistencies, duplication, and difficulty in accessing accurate information, ultimately affecting decision-making and operational efficiency.

Another major gap lies in the limited use of predictive analytics within operational workflows. While machine learning models for lead scoring and churn prediction have been developed and studied extensively, they are often not integrated into day-to-day systems used by brokers. Instead, these models are typically used in isolated analytical environments, making it difficult for sales agents to utilize predictions in real time. This disconnect between analytics and operations reduces the practical impact of machine learning and prevents organizations from fully leveraging their data.

Furthermore, existing systems often lack intuitive and interactive user interfaces that can present complex data in a simplified and actionable format. Many platforms rely on static reports or manual analysis, which requires additional effort and expertise from users. This makes it challenging for brokers to quickly identify high-priority leads or at-risk customers. Additionally, there is limited automation in key areas such as lead prioritization, follow-up scheduling, and proactive customer engagement, resulting in inefficient use of time and resources.

The following table summarizes the key gaps identified in existing systems and how the proposed project addresses them:

Existing System Limitations	Impact	Proposed Solution
Fragmented data across multiple systems	Inconsistent and hard-to-access information	Centralized relational database using Supabase
Lack of real-time analytics	Delayed decision-making	Integration of ML models within the system
Separate CRM and analytics tools	Inefficient workflow	Unified platform combining operations and analytics
No lead prioritization mechanism	Wasted effort on low-quality leads	ML-based lead conversion scoring
No churn prediction system	Poor customer retention	Predictive churn model for early

		identification
Static reports and dashboards	Limited usability	Interactive UI using FlutterFlow
Manual processes for tracking	Increased errors and inefficiency	Automated workflows and data handling

By addressing these gaps, the proposed system aims to bridge the divide between data management and intelligent analytics. It provides a unified, user-friendly platform where machine learning insights are directly integrated into operational workflows. This enables brokers to make timely, informed decisions, improve efficiency, and enhance overall business performance.

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 ASSUMPTIONS AND DEPENDENCIES

The successful design, development, and deployment of the proposed insurance broking system are based on a set of assumptions and dependencies that define the operating environment and constraints of the project. These assumptions help establish a clear understanding of the conditions under which the system is expected to function effectively. It is assumed that the end users of the system, including insurance brokers, sales agents, and administrative personnel, possess basic digital literacy and are familiar with using web-based applications. The system is intended to be accessed through modern web browsers, and therefore, it is assumed that users have access to reliable internet connectivity and compatible devices such as desktops or laptops.

From a technical perspective, the system depends on a centralized backend infrastructure powered by Supabase (PostgreSQL), which is responsible for storing and managing all structured data related to customers, leads, policies, payments, and commissions. It is assumed that the database schema is correctly implemented, normalized, and maintained to ensure data integrity, consistency, and efficient query performance. The frontend of the system is developed using FlutterFlow, which relies on proper configuration and seamless integration with the backend services. The overall functionality of the system assumes stable communication between the frontend and backend components for real-time data retrieval and updates.

Another key assumption is the use of synthetic data for system testing and machine learning model training. Due to the unavailability of real-world insurance datasets, it is assumed that the generated

data accurately reflects real-life scenarios, including customer behavior, payment patterns, and policy lifecycle events. The effectiveness of machine learning models, such as lead conversion prediction and churn analysis, depends on the quality and diversity of this synthetic data. Additionally, the project assumes that the selected machine learning algorithms are sufficient to capture meaningful patterns and provide reasonably accurate predictions within the scope of an academic implementation.

The system also depends on various external tools and technologies, including Python libraries for data generation and model development, and cloud-based services for database hosting. It is assumed that these tools function reliably and are properly configured throughout the development process. Furthermore, the project assumes a controlled environment with a limited number of users and moderate data volume, as it is primarily developed for academic and prototype purposes. Factors such as large-scale deployment, high concurrency, and integration with external insurance APIs or payment gateways are considered beyond the current scope but may influence system behavior in real-world scenarios.

Overall, these assumptions and dependencies provide a foundation for the development of the system, ensuring clarity in design and implementation. While they simplify certain aspects of the project, they also highlight areas that may require further enhancement when transitioning from a prototype to a production-level solution.

3.2 FUNCTIONAL REQUIREMENTS

The functional requirements define the core capabilities and services that the system must provide to support efficient insurance broking operations. These requirements focus on enabling users to manage customers, leads, policies, payments, and commissions while also leveraging machine learning models for predictive analytics. The system is designed to not only handle routine operational tasks but also assist users in making informed decisions through intelligent insights. Each functional module is interconnected, ensuring seamless data flow across different components of the application. By integrating these functionalities into a single platform, the system aims to eliminate manual processes, reduce errors, and improve overall productivity.

3.2.1 Customer and Lead Management

The system shall provide comprehensive functionality for managing customer information and tracking leads throughout their lifecycle. Users should be able to add, update, and view detailed customer profiles, including personal details, contact information, and associated customer groups. The system must support efficient lead creation by capturing relevant information such as interested policy types, expected premium, and assigned sales agents. It should also allow tracking of lead status, including whether a lead is open, converted, or rejected.

In addition, the system should facilitate follow-up management by enabling users to schedule and monitor follow-up activities. It should maintain a history of interactions with customers, allowing sales agents to understand previous communications and make informed decisions. By organizing and prioritizing leads effectively, this module ensures that sales agents can focus on high-potential opportunities and improve conversion rates.

3.2.2 Policy Management

The system shall support end-to-end management of insurance policies, covering the entire lifecycle from policy creation to expiration or renewal. Users should be able to create policies by linking them with customers, insurance companies, and specific policy subtypes. The system must store key details such as policy number, premium amount, start date, end date, and policy status, ensuring accurate and structured record-keeping.

Furthermore, the system should handle different categories of policies, such as Life and Health insurance, each with its own set of attributes. For instance, Life policies may include details like sum assured and nominee information, while Health policies may include sum insured and family coverage details. The module should also support tracking policy status changes, including active, expired, cancelled, or lapsed states. This functionality ensures that brokers can efficiently monitor all policies and maintain accurate records for business operations.

3.2.3 Payment and Commission Management

The system shall enable efficient recording and tracking of all financial transactions related to insurance policies. Users should be able to log payment details such as payment amount, mode, date, and reference number. The system must ensure that all payments are associated with the correct policies and entities, maintaining transparency and accuracy in financial records.

In addition to payment tracking, the system should support commission management for both sales agents and broking agents. It should calculate commissions based on predefined percentages and track their payment status, such as pending or paid. The system must also allow users to view

commission details and generate reports for analysis. This module reduces manual effort, minimizes errors, and ensures proper financial accountability within the organization.

3.2.4 Predictive Churn Analysis

The system shall incorporate a predictive churn analysis feature to identify customers who are at risk of discontinuing their policies. By analyzing historical data such as payment behavior, policy tenure, and claims information, the system assigns a churn risk score to each policy. This score helps categorize customers into different risk levels, such as high, medium, and low.

The system should display these insights through dashboards, allowing users to quickly identify at-risk customers. It must also support filtering and sorting based on risk levels and expiry dates. This enables brokers to take proactive measures, such as contacting customers before policy expiration, thereby improving retention rates and reducing revenue loss. The integration of machine learning into this module enhances the system's ability to provide forward-looking insights.

3.2.5 Lead Conversion Scoring

The system shall provide a lead conversion scoring mechanism to evaluate the likelihood of each lead converting into an active policy. Using machine learning models, the system assigns a conversion score based on various attributes such as customer profile, expected premium, and behavioral patterns. Leads are then categorized into segments such as Hot, Warm, Cool, and Cold based on their scores.

The system should prioritize leads automatically, ensuring that sales agents focus on the most promising opportunities. It must also allow users to view and filter leads based on their scores and categories. This functionality improves efficiency by reducing time spent on low-quality leads and increasing the chances of successful conversions. By integrating predictive analytics into lead management, the system enhances the effectiveness of the sales process.

3.2.6 Dashboard and Reporting

The system shall provide interactive dashboards and reporting features to present key business metrics in a clear and actionable format. Users should be able to view information such as high-risk customers, prioritized leads, active policies, payment summaries, and commission details. The dashboards must support dynamic filtering and sorting options, enabling users to customize their view based on specific requirements.

The reporting module should allow users to generate insights from historical and real-time data, helping them analyze trends and performance. By presenting data visually through charts, tables, and summaries, the system makes it easier for users to interpret complex information. This feature plays a crucial role in supporting data-driven decision-making and improving overall business performance.

3.2.7 Document and Endorsement Management

The system shall support document management by allowing users to upload, store, and retrieve documents related to customers and policies. These documents may include identity proofs, policy documents, and other relevant files. The system should ensure secure storage and easy access to these documents when required.

Additionally, the system must support policy endorsement management, enabling users to track modifications made to policies over time. It should record details of changes, including old and new values, along with timestamps. This ensures transparency and traceability, allowing users to maintain accurate records of policy updates and comply with business requirements.

3.2.8 User and Role Management

The system shall provide user and role management functionalities to ensure secure access and proper authorization. It should allow administrators to create user accounts and assign roles based on responsibilities, such as sales agents, managers, or administrators. The system must implement authentication mechanisms to verify user identity and prevent unauthorized access.

Furthermore, role-based access control should be enforced to restrict access to specific features and data based on user roles. This ensures data security and maintains confidentiality of sensitive information. By managing user permissions effectively, the system supports a secure and organized working environment.

3.3 NONFUNCTIONAL REQUIREMENTS

Nonfunctional requirements define the quality attributes and operational standards that the system must meet to ensure reliability, efficiency, and user satisfaction. Unlike functional requirements, which describe what the system does, nonfunctional requirements focus on how the system performs under various conditions. For an insurance broking application that handles large volumes of sensitive data and supports real-time decision-making, these requirements are critical. They

ensure that the system remains responsive, secure, and dependable while providing a seamless user experience. The following sections outline the key nonfunctional requirements of the system.

3.3.1 Performance Requirements

The system must be designed to deliver efficient and responsive performance under normal and peak usage conditions. It should be capable of handling multiple users simultaneously without significant delays or degradation in performance. Database queries, especially those involving joins and analytical views, must be optimized to ensure quick retrieval of data. Operations such as loading dashboards, filtering records, and displaying reports should occur within an acceptable response time to maintain user productivity.

Additionally, the system should be scalable to accommodate future growth in data volume and user base. As the number of customers, policies, and transactions increases, the system must maintain consistent performance levels. The use of a cloud-based database like Supabase ensures better scalability and performance management. Efficient indexing, caching strategies, and optimized queries should be implemented to reduce latency and improve overall system responsiveness.

3.3.2 Safety Requirements

Safety requirements focus on maintaining the integrity and correctness of data within the system. Since the application deals with critical information such as customer details, financial transactions, and policy records, it is essential to ensure that data is not lost, corrupted, or incorrectly modified. The system must implement validation mechanisms to prevent invalid or inconsistent data from being entered into the database.

Backup and recovery mechanisms should be in place to safeguard data against unexpected failures such as system crashes or network issues. Regular backups ensure that data can be restored to a consistent state in case of any disruptions. Additionally, the system should handle errors gracefully, providing meaningful messages to users without exposing sensitive information. These measures help maintain trust and reliability in the system.

3.3.3 Security Requirements

Security requirements are essential for protecting sensitive data and ensuring safe access to the insurance broking system. Since the application stores confidential information such as customer details, policy records, payment transactions, and commission data, strong security measures must be implemented to prevent unauthorized access, misuse, or data breaches. The system should

provide secure user authentication mechanisms, allowing only verified users to log in and access the application.

Role-based access control must be enforced so that users can only access the modules and information relevant to their responsibilities. For example, sales agents may view leads and customer data, while administrators can manage users and system settings. All communication between the frontend and backend should be secured using encrypted protocols such as HTTPS to protect data during transmission. Sensitive information stored in the database should also be protected through proper encryption and secure storage practices.

The system must maintain activity logs to track important user actions such as login attempts, data updates, and deletions. These logs help in monitoring suspicious activities and improving accountability. Regular database backups, password protection policies, and session management should also be implemented to reduce security risks. By ensuring confidentiality, integrity, and availability of data, these security requirements help build a reliable and trustworthy system for insurance broking operations.

3.3.4 Software Quality Attributes

The system must exhibit high software quality attributes to ensure long-term usability and maintainability. It should be reliable, meaning that it operates consistently without failures under expected conditions. The system should also be user-friendly, with an intuitive interface that allows users to navigate and perform tasks without requiring extensive training.

Maintainability is another important attribute, ensuring that the system can be easily updated or modified as requirements evolve. The use of modular design and clean coding practices facilitates easier debugging and enhancement. Additionally, the system should be scalable, allowing it to handle increasing data and user demands without requiring major redesign. Compatibility across different devices and browsers should also be considered to ensure accessibility for all users.

Overall, these software quality attributes contribute to a robust, efficient, and user-centric system that meets the expectations of modern insurance broking operations.

3.4 SYSTEM REQUIREMENTS

System requirements define the technical environment and resources needed for the successful development, deployment, and operation of the insurance broking application. These requirements ensure that the system functions efficiently, remains scalable, and provides a seamless user experience. They include specifications related to the database, software platforms, and hardware

infrastructure. Proper identification of these requirements helps in selecting the right technologies and ensures that the system can handle real-world usage scenarios effectively.

3.4.1 Database Requirements

The system requires a robust and scalable database to manage large volumes of structured data related to customers, leads, policies, payments, commissions, and analytical outputs. A relational database management system (RDBMS) is essential to maintain data integrity, enforce relationships, and support complex queries. In this project, PostgreSQL is used through Supabase, which provides a cloud-based, secure, and scalable database solution.

The database must support features such as primary and foreign key constraints to ensure data consistency and avoid redundancy. It should allow efficient querying through indexing and optimized joins, especially for generating dashboards and analytical views like churn analysis and lead scoring. Additionally, the database should support real-time data updates and secure API access for seamless integration with the frontend. Backup, recovery, and access control mechanisms are also necessary to ensure data safety and reliability.

3.4.2 Software Requirements (Platform Choice)

The system is developed using a combination of modern software tools and platforms that enable rapid development, scalability, and efficient performance. The frontend is built using FlutterFlow, which allows the creation of responsive and user-friendly web applications with minimal coding effort. It provides seamless integration with backend services and supports features such as real-time data binding, UI customization, and deployment.

For backend and database management, Supabase is used, which is built on PostgreSQL and provides RESTful APIs, authentication, and real-time capabilities. Python is utilized for generating synthetic data and implementing machine learning models for lead conversion scoring and churn prediction. These models are integrated into the system to provide predictive insights. Additionally, version control tools such as Git can be used to manage code changes, and modern browsers are required to run the application efficiently.

3.4.3 Hardware Requirements

The hardware requirements for this system are minimal, as it is primarily a web-based application hosted on cloud infrastructure. Users can access the system using standard computing devices such as desktops, laptops, or tablets with internet connectivity. A system with at least 4 GB RAM, a

modern processor, and sufficient storage is recommended to ensure smooth operation of the application and browser-based interface.

On the server side, since the database and backend services are hosted on cloud platforms like Supabase, there is no need for dedicated physical servers. The cloud infrastructure handles scalability, storage, and performance optimization. However, stable internet connectivity is essential for both development and usage of the system. For development purposes, a machine capable of running development tools such as FlutterFlow (browser-based) and Python environments is required.

3.5 Analysis Models: SDLC Model to be Applied

The development of this project follows the Agile Software Development Life Cycle (SDLC) model. Agile methodology is chosen due to its flexibility, iterative nature, and ability to adapt to changing requirements. Instead of following a rigid sequence of steps, the project is developed in smaller increments, allowing continuous improvement and refinement of features.

In this approach, the project is divided into multiple phases such as requirement analysis, system design, development, testing, and deployment. Each phase is revisited in cycles, enabling feedback incorporation and quick resolution of issues. For example, initial versions of the database and frontend are developed and tested, followed by integration of machine learning models and dashboard features. Regular testing ensures that each component functions correctly and meets the defined requirements.

The Agile model also promotes collaboration and adaptability, allowing developers to make improvements based on testing results and user feedback. This ensures that the final system is more reliable, user-friendly, and aligned with project objectives. By adopting this methodology, the project achieves a balance between structured development and flexibility, making it suitable for modern application development.

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The system architecture of the proposed insurance broking application is designed using a **modular, scalable, and layered approach** to ensure efficient performance, maintainability, and ease of integration. The architecture follows a **three-tier structure**, consisting of the presentation layer (frontend), application layer (backend), and data layer (database). This separation of concerns allows each component of the system to function independently while maintaining seamless communication across layers. Such an architecture not only improves system reliability but also makes it easier to scale and enhance the system as business requirements evolve. (Fig. 3, 4)

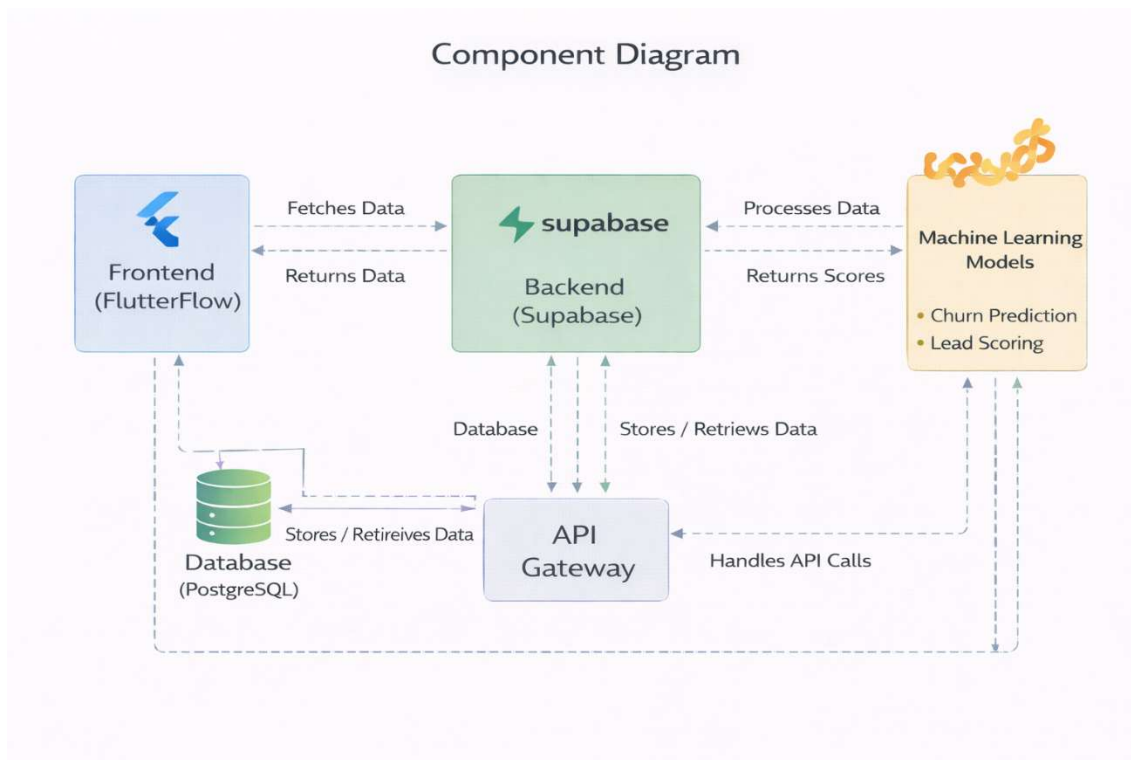


Fig. 3 Component Diagram

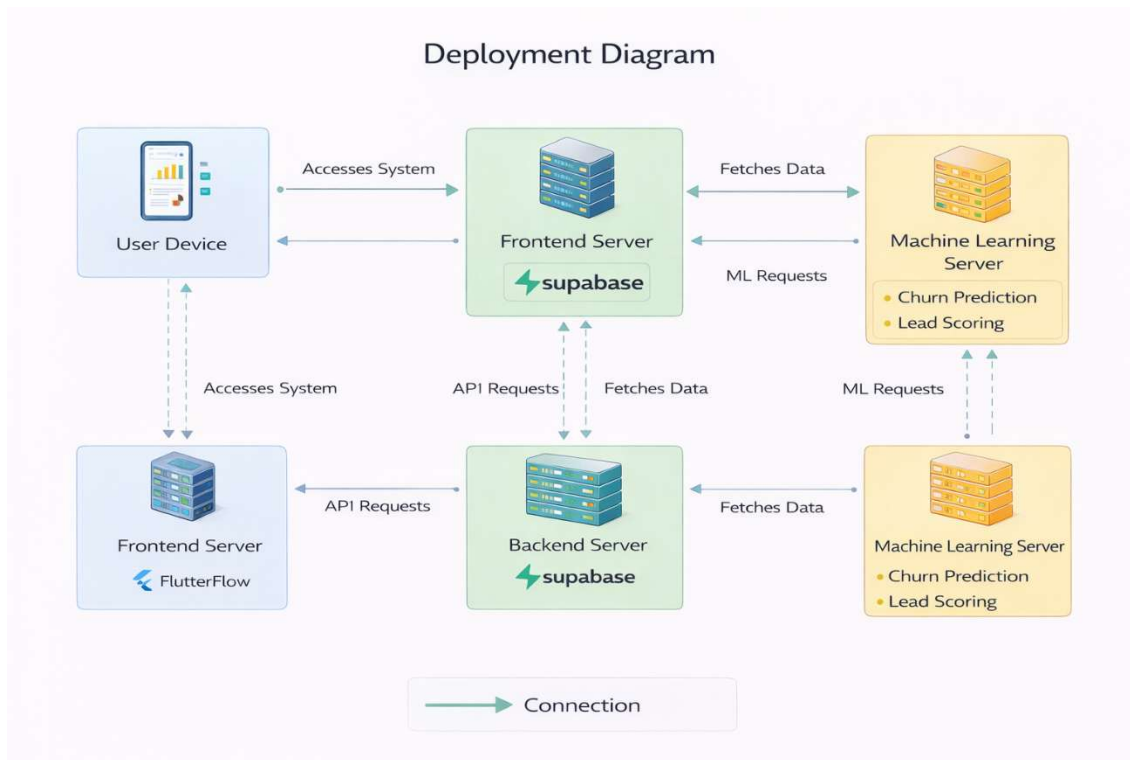


Fig. 4 Deployment Diagram

The **presentation layer** is developed using FlutterFlow, which provides a responsive and user-friendly interface accessible through web browsers. This layer is responsible for interacting with users such as insurance brokers, sales agents, and administrators. It displays dashboards, reports, and data tables, and allows users to perform operations such as managing customers, tracking leads, viewing policy details, and analyzing predictive insights. The interface is designed to be intuitive, ensuring that users can easily navigate through the system without requiring extensive technical knowledge. Features such as filtering, sorting, and real-time updates enhance usability and improve the overall user experience.

The **application layer**, powered by Supabase, acts as the intermediary between the frontend and the database. It handles business logic, API communication, authentication, and data processing. Supabase provides RESTful APIs and real-time capabilities, enabling smooth interaction between the user interface and the backend services. This layer ensures that user requests are validated, processed, and securely transmitted to the database. It also manages user authentication and role-based access control, ensuring that only authorized users can access specific features and data. By centralizing application logic, this layer enhances system security and consistency.

The **data layer** consists of a PostgreSQL database hosted on Supabase, which stores all structured data related to customers, leads, policies, payments, commissions, and documents. The database is designed using relational modeling techniques, ensuring data integrity through the use of primary and foreign key constraints. It supports complex queries and joins required for generating dashboards and analytical reports. Additionally, the database stores outputs from machine learning models, such as churn risk scores and lead conversion probabilities, which are used to provide actionable insights to users.

An important component of the system architecture is the integration of **machine learning models**, developed using Python. These models operate as an analytical layer that processes historical data to generate predictions. The churn prediction model identifies customers who are at risk of policy lapse, while the lead scoring model evaluates the likelihood of lead conversion. The outputs from these models are stored in the database and displayed on the frontend, enabling users to make data-driven decisions. This integration transforms the system from a traditional data management tool into an intelligent decision-support system.

The architecture also emphasizes **scalability and flexibility**, allowing the system to handle increasing volumes of data and users. Since the backend and database are hosted on cloud infrastructure, the system can scale dynamically without requiring significant changes to the architecture. Furthermore, the modular design ensures that new features, such as additional analytics modules or integrations with external systems, can be added with minimal impact on existing components.

In summary, the system architecture provides a well-structured and efficient framework for the insurance broking application. By combining a user-friendly frontend, a robust backend, a reliable database, and intelligent machine learning models, the architecture ensures seamless data flow, secure operations, and enhanced decision-making capabilities. This design not only addresses current business requirements but also lays a strong foundation for future growth and technological advancements. (Fig. 5)

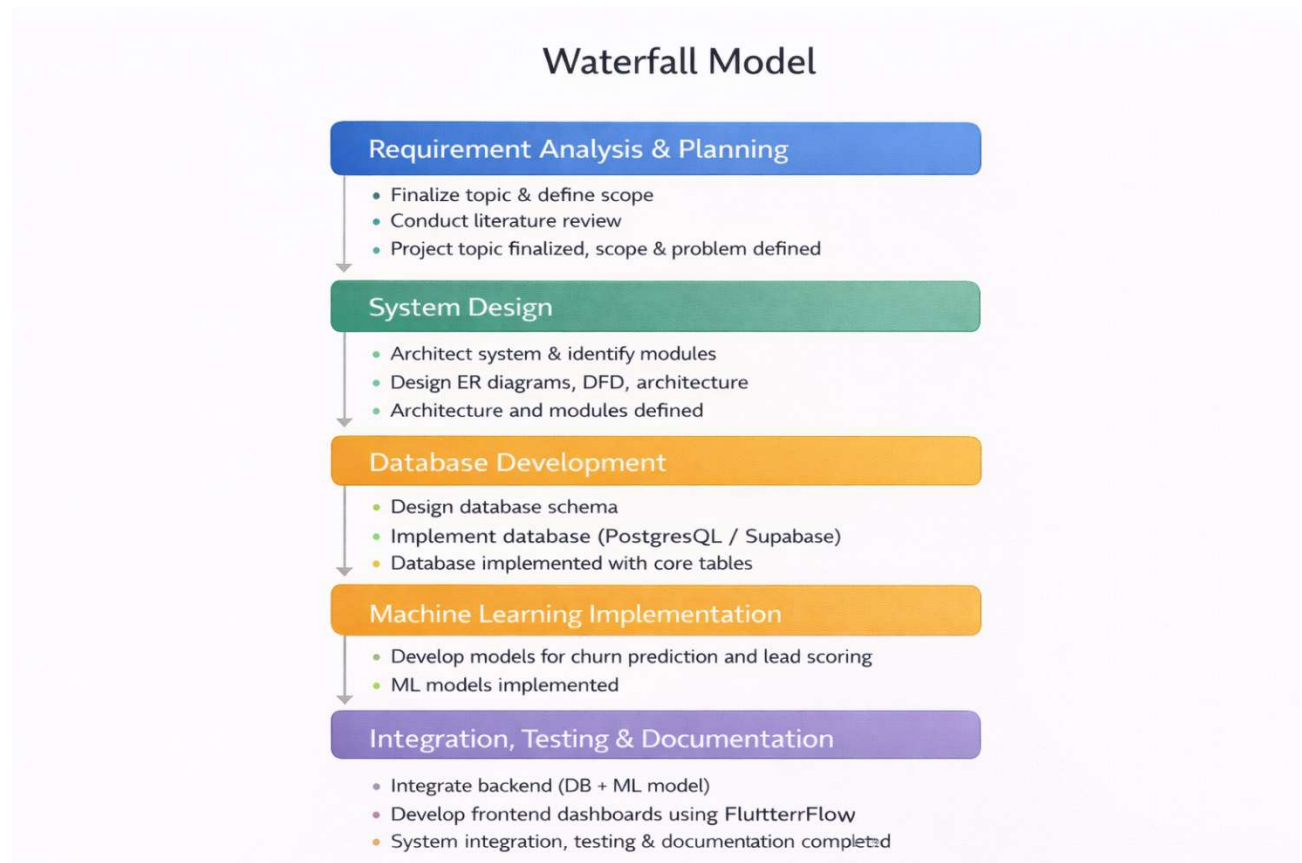


Fig. 5 Waterfall Model

4.2 MATHEMATICAL MODEL

The mathematical model of the proposed system represents the analytical framework used to transform insurance data into meaningful predictions. The system is primarily based on **supervised machine learning models** for two major tasks: **customer churn prediction** and **lead conversion scoring**. These models use historical data to estimate probabilities and classify outcomes, enabling data-driven decision-making for insurance brokers.

4.2.1 System Representation

The system can be mathematically represented as:

$$S = \{I, P, O\}$$

Where:

- **I (Input):** Raw insurance and customer data
- **P (Process):** Data preprocessing and machine learning computation
- **O (Output):** Predicted scores and classifications

4.2.2 Input (I)

The input consists of structured data collected from the database. Machine learning models rely on multiple features derived from customer behavior and policy details.

$$\mathbf{I} = \{\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3, \dots, \mathbf{X}_n\}$$

Where each $\mathbf{X}_i$ represents features such as:

- Policy tenure
- Payment status
- Customer demographics
- Interaction history
- Lead attributes (expected premium, category, etc.)

Machine learning models require numerical input; hence categorical data is transformed using encoding techniques before processing.

4.2.3 Process (P)

The process consists of multiple stages:

Data Preprocessing

- Handling missing values
- Encoding categorical variables
- Feature selection and normalization

Churn Prediction Model

Customer churn prediction is treated as a **binary classification problem**, where:

- $\mathbf{Y} = 1 \rightarrow$ Customer will churn
- $\mathbf{Y} = 0 \rightarrow$ Customer will not churn

The model predicts:

$$\mathbf{P}(\text{Churn}) = \mathbf{f}(\mathbf{X})$$

Where:

- $\mathbf{X}$ = input feature vector
- $\mathbf{f}$ = machine learning function (e.g., Random Forest, Logistic Regression, XGBoost)

Logistic Regression formulation:

$$\mathbf{P}(\mathbf{Y}=1|\mathbf{X})=\frac{1}{1+e^{\{-(w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n)\}}}$$

Machine learning models analyze historical patterns to identify customers likely to leave, enabling proactive retention strategies.

Lead Conversion Model

Lead conversion is modeled as a **probability estimation problem**, where:

- Output = Probability that a lead converts into a policy

$$P(\text{Conversion}) = f(X)$$

Using Logistic Regression:

$$P(\text{Conversion}) = \frac{1}{1 + e^{-z}}, \quad z = w_1x_1 + w_2x_2 + \dots + w_nx_n$$

This model ranks leads based on their likelihood of conversion, helping prioritize high-value opportunities.

4.2.4 Output (O)

The system produces the following outputs:

- **Churn Probability Score (0–1)**
- **Risk Tier (High / Medium / Low)**
- **Conversion Probability (%)**
- **Lead Priority Ranking**

Mathematically:

$$O = \{C_s, R_t, L_s, P_r\}$$

Where:

- C_s = Churn Score
- R_t = Risk Tier
- L_s = Lead Score
- P_r = Priority Rank

4.2.5 Model Optimization

Machine learning models are evaluated using performance metrics such as:

- Accuracy
- Precision and Recall
- ROC-AUC Score

Studies show that models like **Random Forest** and **XGBoost** often perform well for churn prediction in insurance datasets.

4.2.6 Assumptions

- Historical data reflects real customer behavior
- Input data is properly cleaned and structured
- Features selected are relevant to prediction

4.2.7 Constraints

- Model accuracy depends on data quality
- Limited real-time data availability
- Model requires periodic retraining

4.2.8 Conclusion

The mathematical model provides a structured approach for analyzing insurance data using machine learning techniques. By estimating churn probability and lead conversion likelihood, the system enables proactive decision-making, improves customer retention, and optimizes sales efforts. This data-driven approach significantly enhances the efficiency and effectiveness of insurance broking operations.

4.3 DATA FLOW DIAGRAMS

Data Flow Diagrams (DFDs) (Fig.6, 7, 8) represent the movement of data within the system and illustrate how different components interact with each other. The system involves multiple entities such as users, database, and machine learning modules.

At a high level, the data flow begins with users entering or updating information such as customer details, leads, and policy data through the frontend interface. This data is then sent to the backend (Supabase), where it is processed and stored in the database. Machine learning models access this data to generate predictions such as lead conversion scores and churn risk values. These outputs are then stored back in the database and displayed on the dashboard for users.

The DFD ensures clarity in understanding how data is processed, stored, and utilized across the system. It highlights the interaction between different modules and helps in identifying dependencies and data flow paths.

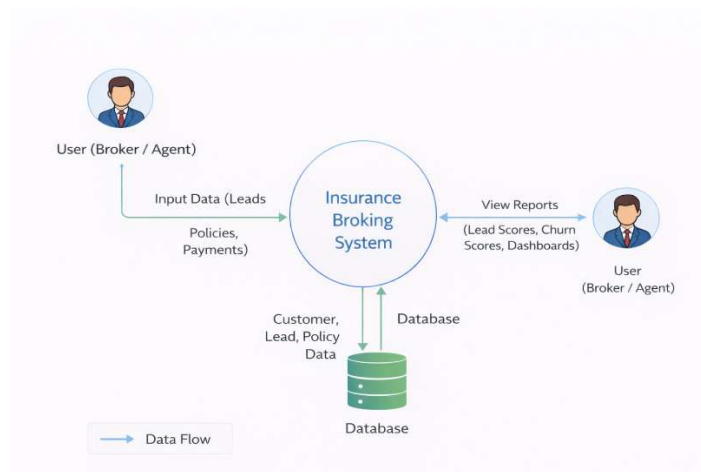


Fig. 6 DFD Level – 0

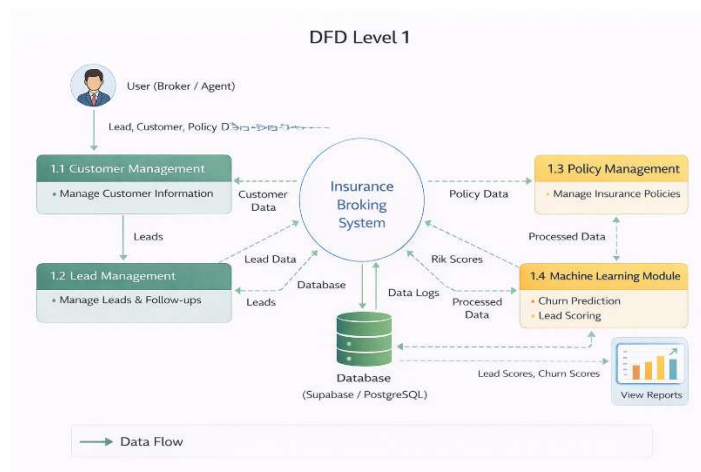


Fig. 7 DFD Level – 1

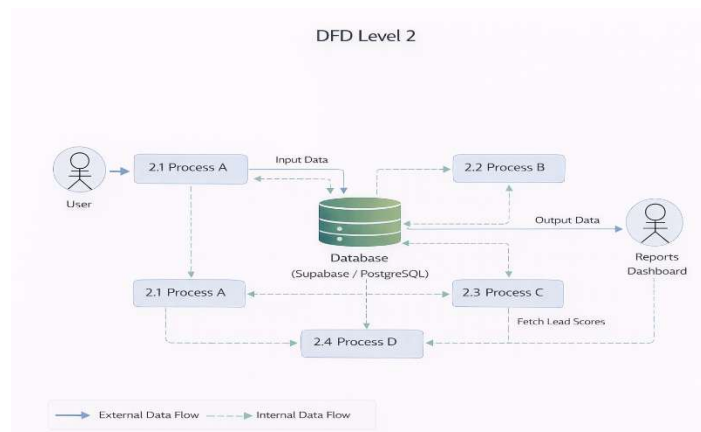


Fig. 8 DFD Level – 2

The Entity Relationship (ER) diagram (Fig. 9) provides a visual representation of the database structure and the relationships between different entities. In this system, key entities include customers, leads, policies, payments, commissions, insurance companies, and agents.

The ER diagram helps in understanding how data is organized and ensures that relationships are correctly implemented to maintain data integrity. It also aids in designing efficient queries and optimizing database performance.



Unified Modeling Language (UML) diagrams are used to represent the structure and behavior of the system from different perspectives. Various UML diagrams are utilized in this project to provide a comprehensive understanding of system functionality.

Use Case Diagrams (Fig. 10) illustrate the interaction between users (such as brokers and sales agents) and the system, showing the functionalities they can perform, such as managing leads,

viewing dashboards, and handling policies. Class Diagrams (Fig. 11) represent the structure of the system by defining classes, attributes, and relationships between different components. Sequence Diagrams (Fig. 12) show the flow of interactions between different system components over time, such as how data moves from the frontend to the backend and back to the user.

These UML diagrams provide a clear and standardized way to visualize system design, making it easier to understand, communicate, and implement the overall architecture. (Fig. 13)

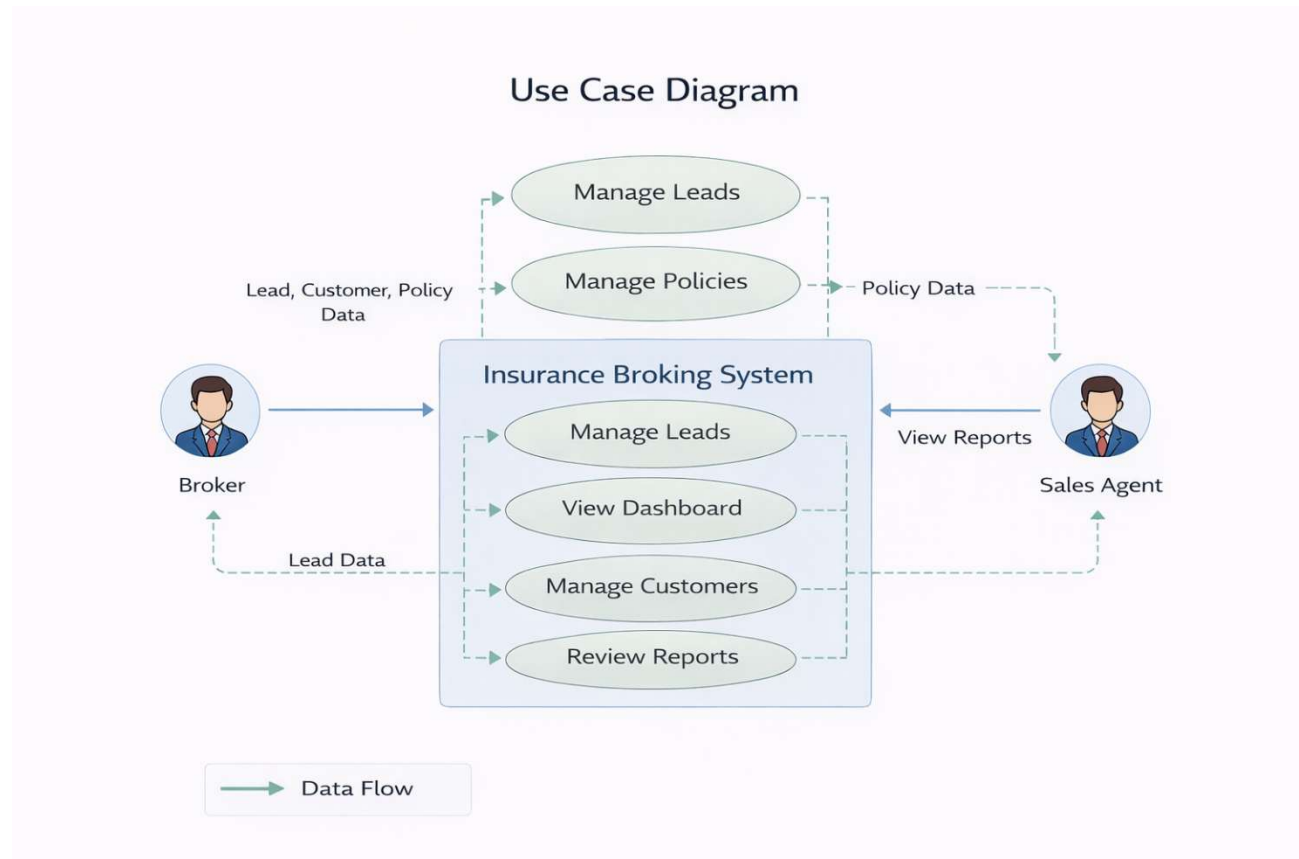


Fig. 10 Use Case Diagram

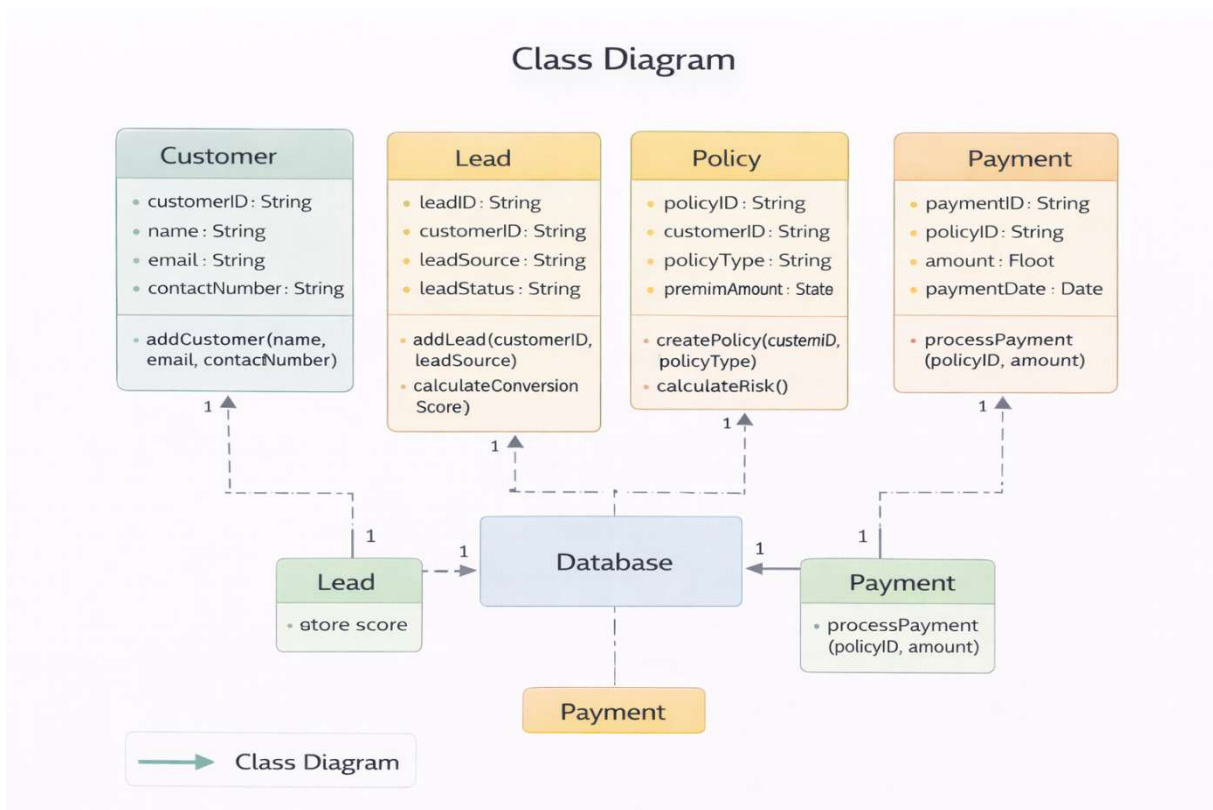


Fig. 11 Class Diagram

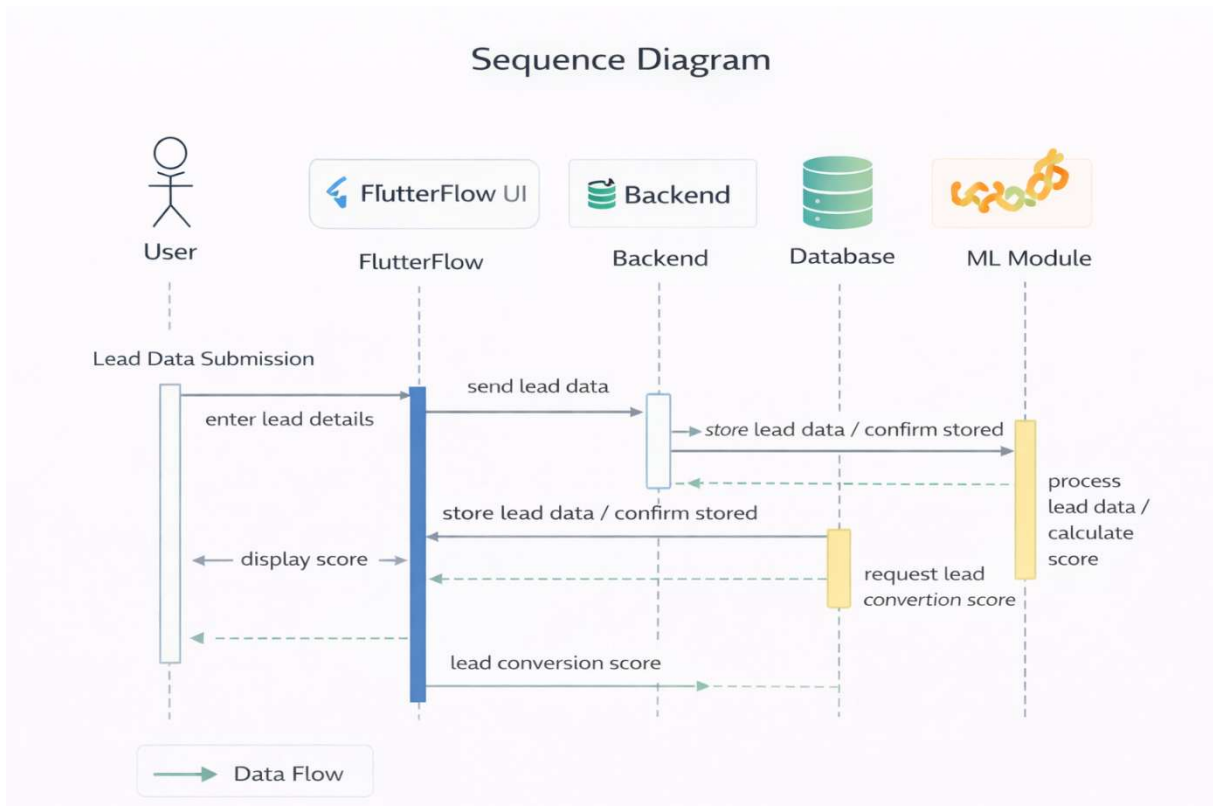


Fig. 12 Sequence Diagram

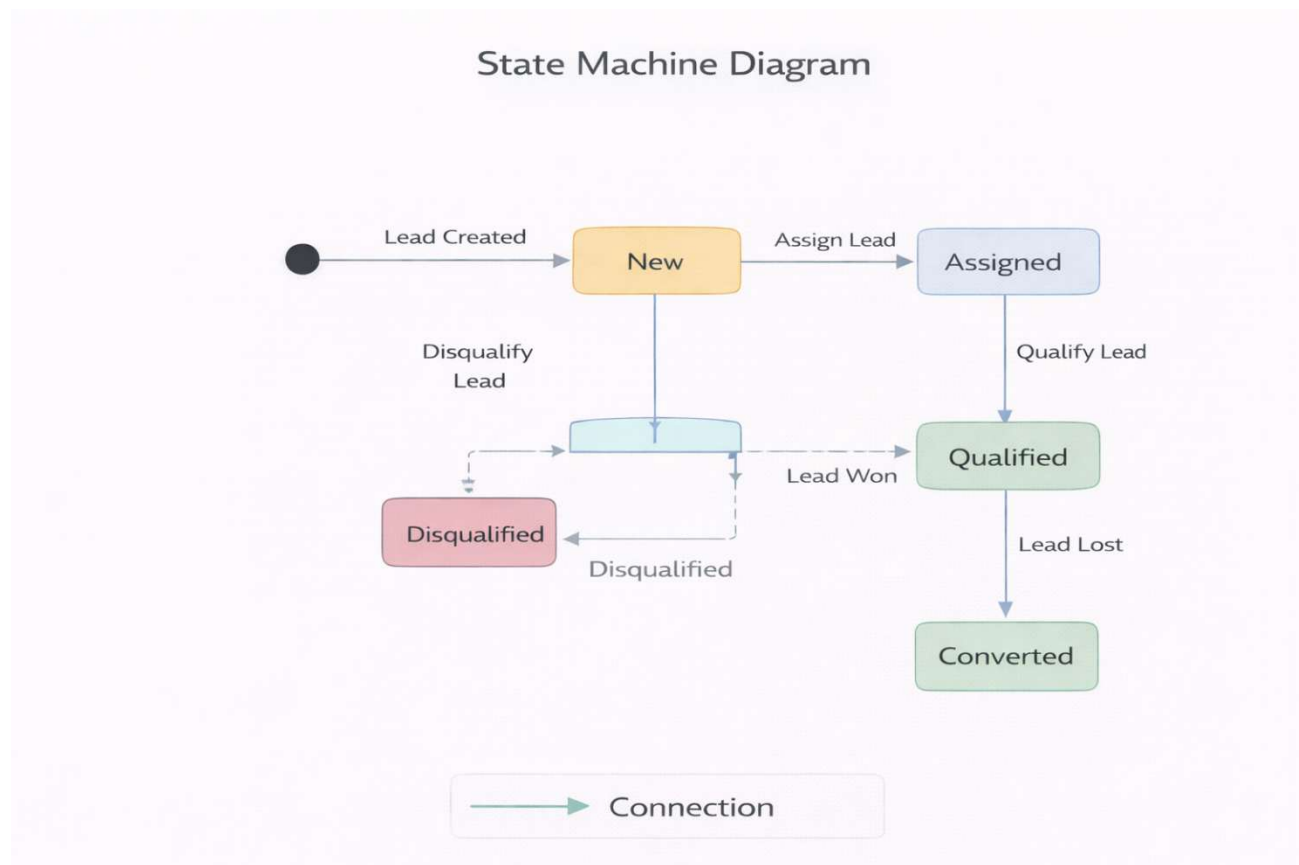


Fig. 13 State Machine Diagram

PROJECT PLAN

5.1 PROJECT ESTIMATE

The project estimate provides an overview of the time, effort, and resources required to successfully complete the development of the insurance broking system. The estimation is based on the different phases of the Software Development Life Cycle (SDLC), including requirement analysis, design, development, testing, and deployment. Each phase is allocated a specific duration depending on its complexity and importance in the overall project.

The project is developed as a team-based academic project, where tasks are distributed among team members to ensure efficient progress. The estimation considers factors such as database design complexity, frontend development using FlutterFlow, integration of machine learning models, and testing of system functionalities. Since the project involves both development and analytical components, additional time is allocated for model training, validation, and integration with the application.

The estimated effort also includes time for debugging, refinement, and documentation. Iterative development is followed, allowing continuous improvement and adjustments based on testing

results. Overall, the project is planned to be completed within a defined academic timeline, ensuring that all core functionalities and features are implemented effectively.

The approximate effort distribution is as follows:

- Requirement Analysis: Understanding project scope and defining system requirements
- System Design: Creating database schema, ER diagrams, and architecture
- Development: Implementing frontend, backend, and database integration
- Machine Learning Integration: Developing and integrating predictive models
- Testing and Debugging: Ensuring system accuracy and performance
- Documentation: Preparing reports, diagrams, and final submission

5.2 PROJECT SCHEDULE - TIMELINE CHART

The project schedule outlines the timeline for completing different phases of the project in a structured and organized manner. It ensures that all activities are completed within the given timeframe and helps track progress throughout the development process. The schedule is designed in phases, with each phase building upon the previous one.

The project begins with requirement analysis and planning, followed by system design and database creation. The development phase includes frontend and backend implementation, along with integration of machine learning models. This is followed by testing, debugging, and final deployment of the system. Proper time allocation is ensured for each phase to avoid delays and maintain quality.

The timeline can be represented as follows:

Phase	Activity	Duration
Phase 1	Requirement Analysis & Planning (Finalize topic, define scope, literature review)	2 Weeks (Week 1–2)
Phase 2	System Design (System architecture design, module identification, ER diagram, DFD)	2 Weeks (Week 3–4)
Phase 3	Database Development (Database implementation using PostgreSQL / Supabase)	1 Week (Week 5)
Phase 4	Data Preparation (Synthetic dataset generation, preprocessing, feature engineering)	2 Weeks (Week 6–7)
Phase 5	Machine Learning Implementation (Model development, evaluation, and optimization)	2 Weeks (Week 8–9)

Phase	Activity	Duration
Phase 6	System Integration (Backend integration with database and ML models)	1 Week (Week 10)
Phase 7	Frontend Development (Dashboard development using FlutterFlow)	1 Week (Week 11)
Phase 8	Testing & Documentation (System testing, validation, and final documentation)	1 Week (Week 12)

This timeline ensures a systematic approach to project execution, allowing sufficient time for development, testing, and refinement. It also helps in monitoring progress and ensuring that the project is completed successfully within the planned duration. (Fig. 14)

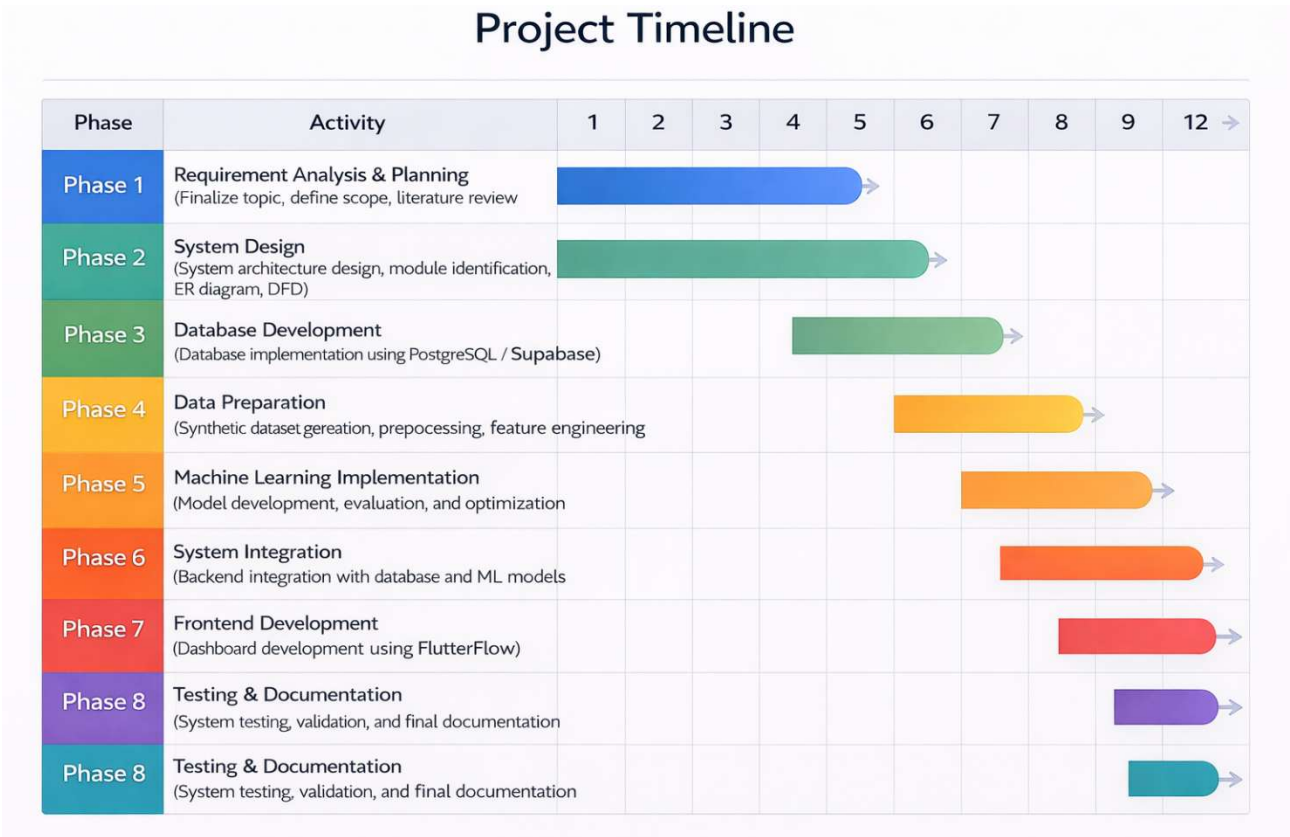


Fig. 14 Timeline Chart

PROJECT IMPLEMENTATION

6.1 OVERVIEW OF PROJECT MODULES

The proposed system is designed as a modular and scalable insurance broking application, where each module is responsible for handling a specific aspect of the overall workflow. This modular approach ensures better organization, easier maintenance, and efficient system performance. The application integrates operational functionalities with predictive analytics, enabling both data management and intelligent decision-making.

The system is divided into the following key modules:

- **Customer Management Module:** Handles customer data, including personal details, contact information, and customer segmentation. It ensures that all customer-related information is stored and managed efficiently.
- **Lead Management Module:** Manages the lifecycle of leads, from creation to conversion or rejection. It includes features such as lead assignment, follow-up tracking, and status updates.
- **Policy Management Module:** Responsible for managing insurance policies, including policy creation, updates, and tracking. It maintains details such as policy type, premium, duration, and status.
- **Payment and Commission Module:** Tracks financial transactions, including premium payments and commissions for agents. It ensures transparency and accurate financial record-keeping.
- **Churn Prediction Module:** Uses machine learning to identify customers at risk of not renewing their policies. It assigns risk scores and categorizes customers for proactive retention strategies.
- **Lead Scoring Module:** Predicts the likelihood of lead conversion using machine learning techniques. It prioritizes leads based on their conversion probability.
- **Dashboard and Reporting Module:** Provides visual insights into key metrics such as high-risk customers, lead performance, and policy statistics. It supports data-driven decision-making.

Each module interacts with others through a centralized database, ensuring seamless data flow and integration across the system.

6.2 TOOLS AND TECHNOLOGIES USED

The development of the system utilizes a combination of modern tools and technologies to ensure efficiency, scalability, and ease of implementation. These technologies are selected based on their ability to support both application development and machine learning integration.

- **FlutterFlow:** Used for frontend development, enabling rapid creation of responsive and interactive user interfaces. It allows seamless integration with backend services and supports real-time data binding.
- **Supabase (PostgreSQL):** Serves as the backend and database platform. It provides a secure, scalable, and cloud-based relational database along with APIs and authentication features.
- **Python:** Used for data generation, preprocessing, and implementation of machine learning models such as churn prediction and lead scoring.
- **Machine Learning Libraries:** Libraries such as Scikit-learn are used to build and evaluate predictive models like Logistic Regression, Random Forest, and Gradient Boosting.
- **Git (Version Control):** Used to manage code changes and maintain version history, ensuring collaborative and organized development.
- **Web Browser:** Required to run and test the FlutterFlow application, ensuring accessibility across different devices.

These technologies collectively enable the development of a robust, efficient, and intelligent insurance broking system.

6.3 ALGORITHM DETAILS

The system incorporates machine learning algorithms to enhance decision-making through predictive analytics. These algorithms are applied to key business problems such as customer churn prediction and lead conversion scoring.

6.3.1 Algorithm 1: Predictive Renewal Churn Model

The Predictive Renewal Churn Model is designed to identify customers who are likely to discontinue or not renew their insurance policies. This model uses supervised learning techniques to analyze historical data and generate a churn risk score.

The input features for the model include payment status, policy tenure, claims amount, and other behavioral indicators. The model processes this data to identify patterns associated with customer churn. Algorithms such as Random Forest or Gradient Boosting are used due to their ability to handle complex relationships and provide accurate predictions.

The output of the model is a numerical risk score, which is further categorized into risk levels such as High, Medium, and Low. These results are stored in the database and displayed on the dashboard, allowing brokers to take proactive actions such as contacting at-risk customers before policy expiration.

6.3.2 Algorithm 2: Lead Conversion Scoring Model

The Lead Conversion Scoring Model is used to predict the probability of a lead converting into an active policy. This model helps sales agents focus on high-potential leads, thereby improving efficiency and conversion rates.

The model takes input features such as customer profile, expected premium, demographic attributes, and interaction history. Logistic Regression or ensemble methods are used to calculate the probability of conversion. The output is a conversion score, typically expressed as a percentage.

Based on this score, leads are categorized into segments such as Hot, Warm, Cool, and Cold. These categories help prioritize leads, ensuring that sales agents allocate their efforts effectively. The results are integrated into the dashboard, enabling real-time decision-making.

6.3.3 Additional Notes

Both models are trained using historical or synthetic data and are integrated into the system for prediction purposes. The outputs are stored in the database and utilized by the frontend for visualization and analysis. This integration of machine learning enhances the system's capability to provide intelligent insights, making it more effective than traditional manual systems.

RESULTS

7.1 OUTCOMES

The proposed system successfully provides a centralized platform for managing customers, leads, and insurance policies in an organized and efficient manner. It replaces manual processes with a structured digital solution, improving overall workflow and data accessibility.

The implementation of machine learning models enables churn prediction and lead conversion scoring, helping brokers identify high-risk customers and prioritize potential leads. This supports proactive decision-making, improves customer retention, and increases conversion efficiency.

Additionally, the system includes an interactive dashboard that presents key insights in a clear and user-friendly format. Overall, the project delivers a data-driven solution that enhances operational efficiency and business performance in insurance broking.

7.2 SCREENSHOTS

Predictive Renewal Churn Model

```
import joblib
import pandas as pd
from supabase import create_client
from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Get these from Supabase Dashboard → Project Settings → API
SUPABASE_URL = "--"
SUPABASE_KEY = "--" #Not shown for security reasons.

FEATURES = [
    "tenure_days", "days_to_expiry", "premium_amount",
    "payment_count", "avg_payment_amount", "days_to_first_payment",
    "endorsement_count", "customer_policy_count",
    "is_group_customer", "tenure_years",
]

def load_data():
    client = create_client(SUPABASE_URL, SUPABASE_KEY)

    # Fetch in pages (REST API has a 1000-row default limit)
    all_rows = []
    page_size = 1000
    offset = 0
    while True:
        response = (
            client.from_("churn_features")
            .select("*,")
            .not_("is_churned", "null")
            .range(offset, offset + page_size - 1)
            .execute()
        )
        rows = response.data
        all_rows.extend(rows)
        if len(rows) < page_size:
            break
        offset += page_size

    df = pd.DataFrame(all_rows)
    print(f"Fetches {len(df)} rows")

    df["is_life"] = (df["policy_category"] == "Life").astype(int)
    features = FEATURES + ["is_life"]
    df[features] = df[features].fillna(0)
    X = df[features]
    y = df["churned"].astype(int)
    return X, y

def train():
    X, y = load_data()
    print(f"Dataset: {len(X)} rows | Churn rate: {y.mean():.1%}")

    model = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", XGBClassifier(
            n_estimators=500,
            max_depth=4,
            learning_rate=0.04,
            subsample=0.8,
            colsample_bytree=0.8,
            min_child_weight=5,
            eval_metric="auc",
            random_state=42,
        )),
    ])

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    aucs = cross_val_score(model, X, y, cv=cv, scoring="roc_auc")
    print(f"CV AUC: {aucs.mean():.4f} ± {aucs.std():.4f}")

    model.fit(X, y)

    clf = model.named_steps["clf"]
    fi = pd.Series(clf.feature_importances_, index=X.columns).sort_values(ascending=False)
    print(f"\nTop features:\n", fi.head(8))

    joblib.dump(model, "churn_model.pkl")
    joblib.dump(FEATURES + ["is_life"], "feature_names.pkl")
    print(f"\nSaved → churn_model.pkl")

if __name__ == "__main__":
    train()
```

Fetches 5019 rows

Dataset: 5019 rows | Churn rate: 48.7%
CV AUC: 0.8640 ± 0.0142

Top features:

tenure_years	0.450338
days_to_expiry	0.253567
tenure_days	0.140774
days_to_first_payment	0.043790
endorsement_count	0.025104
avg_payment_amount	0.023353
premium_amount	0.022105
payment_count	0.021268
dtype: float32	

Saved → churn_model.pkl

Fig. 15 Churn Model

Lead Conversion & Scoring

```

1 import { createClient } from "https://esm.sh/supabase/supabase.js@2";
2
3 const supabaseUrl = Deno.env.get("SUPABASE_URL");
4 const supabaseKey = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY");
5
6 const JSON_HEADERS = { "Content-Type": "application/json" };
7
8 const WEIGHTS = {
9   intercept: -1.2,
10  has_aadhar: 0.6,
11  has_pan: 0.6,
12  has_active_policy: 1.1,
13  prior_policy_per_count: 0.3,
14  customer_group_corporate: 0.9,
15  customer_group_sme: 0.5,
16  insurance_category_life: 0.4,
17  insurance_category_health: 0.3,
18  expected_premium_normalized: 0.7,
19  has_assigned_agent: 0.4,
20  follow_up_very_soon: 0.5,
21  follow_up_soon: 0.2,
22  follow_up_overdue: -0.5,
23  lead_age_decay_per_day: -0.01,
24 };
25
26 function sigmoid(x: number): number {
27   return 1 / (1 + Math.exp(-x));
28 }
29
30 function getLabel(score: number): string {
31   if (score >= 0.75) return "Hot";
32   if (score >= 0.50) return "Warm";
33   if (score >= 0.25) return "Cool";
34   return "Cold";
35 }
36
37 function scoreLead(lead: Record<string, unknown>): number {
38   let logit = WEIGHTS.intercept;
39
40   if (lead.has_aadhar) logit += WEIGHTS.has_aadhar;
41   if (lead.has_pan) logit += WEIGHTS.has_pan;
42
43   if (lead.has_active_policy) logit += WEIGHTS.has_active_policy;
44   const priorCount = Math.min(Number(lead.prior_policy_count || 0), 3);
45   logit += priorCount * WEIGHTS.prior_policy_per_count;
46
47   const group = String(lead.customer_group_name || "").toLowerCase();
48   if (group.includes("corp") || group.includes("enterprise"))
49     logit += WEIGHTS.customer_group_corporate;
50   else if (group.includes("sme") || group.includes("business"))
51     logit += WEIGHTS.customer_group_sme;
52
53   const category = String(lead.insurance_category || "").toLowerCase();
54   if (category === "life") logit += WEIGHTS.insurance_category_life;
55   if (category === "health") logit += WEIGHTS.insurance_category_health;
56
57   if (lead.expected_premium && Number(lead.expected_premium) > 0) {
58     const norm = Math.min(Number(lead.expected_premium) / 100000, 1);
59     logit += norm * WEIGHTS.expected_premium_normalized;
60   }
61
62   if (lead.has_assigned_agent) logit += WEIGHTS.has_assigned_agent;
63
64   const daysToFollowUp = Number(lead.days_to_follow_up ?? 999);
65   if (lead.is_follow_up_overdue) logit += WEIGHTS.follow_up_overdue;
66   else if (daysToFollowUp <= 3) logit += WEIGHTS.follow_up_very_soon;
67   else if (daysToFollowUp <= 7) logit += WEIGHTS.follow_up_soon;
68
69   const daysSinceCreated = Number(lead.days_since_created || 0);
70   logit += Math.max(daysSinceCreated * WEIGHTS.lead_age_decay_per_day, -0.8);
71
72   return sigmoid(logit);
73 }
74
75 Deno.serve(async (req) => {
76   try {
77     if (!supabaseUrl || !supabaseKey) {
78       return new Response(
79         JSON.stringify({ error: "Missing env vars" }),
80         { status: 500, headers: JSON_HEADERS }
81       );
82     }
83
84     const supabase = createClient(supabaseUrl, supabaseKey);
85     const body = await req.json().catch(() => ({}));
86     const singleLeadId = body?.lead_id;
87
88     // If single lead, score just that one
89     if (singleLeadId) {
90       const { data: leads, error } = await supabase
91         .from("leads_with_scoring_features")
92         .select("")
93         .eq("lead_id", singleLeadId);
94
95       if (error) return new Response(JSON.stringify({ error: error.message }), { status: 500, headers: JSON_HEADERS });
96
97       for (const lead of leads || []) {
98         const score = scoreLead(lead);
99         await supabase.from("leads").update({
100           conversion_score: Math.round(score * 1000) / 1000,
101           score_label: getLabel(score),
102           scored_at: new Date().toISOString(),
103         }).eq("id", lead.lead_id);
104       }
105
106       return new Response(JSON.stringify({ scored: leads?.length ?? 0 }), { headers: JSON_HEADERS });
107     }
108
109     // Batch mode: score unscored leads only, 200 at a time
110     const BATCH_SIZE = 500;
111     const { data: leads, error } = await supabase
112       .from("leads_with_scoring_features")
113       .select("")
114       .is("scored_at", null)
115       .limit(BATCH_SIZE);
116
117     if (error) return new Response(JSON.stringify({ error: error.message }), { status: 500, headers: JSON_HEADERS });
118
119     let scored = 0;
120     for (const lead of leads || []) {
121       const score = scoreLead(lead);
122       const { error: updateError } = await supabase
123         .from("leads")
124         .update({
125           conversion_score: Math.round(score * 1000) / 1000,
126           score_label: getLabel(score),
127           scored_at: new Date().toISOString(),
128         })
129         .eq("id", lead.lead_id);
130
131       if (!updateError) scored++;
132     }
133
134     return new Response(
135       JSON.stringify({
136         scored,
137         message: scored === BATCH_SIZE
138           ? "Scored " + scored + " leads. Call again to score next batch."
139           : "Done! Scored " + scored + " leads in this batch."
140       }),
141       { headers: JSON_HEADERS }
142     );
143   } catch (err) {
144     return new Response(
145       JSON.stringify({ step: "unexpected", error: String(err), stack: err?.stack }),
146       { status: 500, headers: JSON_HEADERS }
147     );
148   }
149 });

```

Fig. 16 Lead Conversion

Flutter Flow

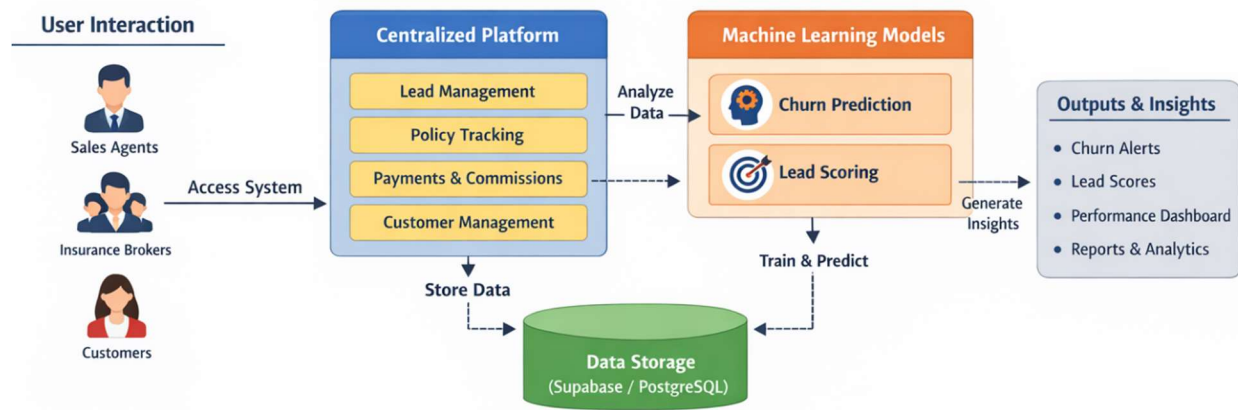


Fig. 17 Work-Flow

Insurvia

Menu

- Dashboard
- Customer List
- Sales Agent
- Insurance Companies
- Churn View
- Leads
- Profile

Policy
Policy Lists

Customers
Total count of Customers
1000

Policies
Total count of Policies
1000

Customer Name	Broker Name	Policy Type	Policy Name	Issue Date	Start Date	End Date	Premium Amount	Sum Assured	Payment Status	Policy Status
Azaan Kunda	Hiral Randhawa	Life	policy_name	2026-03-20 16:34:12.218650	2025-11-02 00:00:00.000	2028-11-01 00:00:00.000	52889	1057780	Paid	Active
Jagvi Munshi	Ati Thaman	Health	Niva Bupa ReAssure Plan	2026-03-20 16:34:12.218650	2025-01-31 00:00:00.000	2026-01-31 00:00:00.000	15651	125208	Paid	Lapsed
Darika Munshi	Tanveer Mandal	Life	ICIPI Protect Smart	2026-03-20 16:34:12.218650	2026-03-01 00:00:00.000	2027-03-01 00:00:00.000	63727	1147086	Paid	Active
Champak Pandit	Vanya Savant	Health	Care Health Joy Plan	2026-03-20 16:34:12.218650	2023-08-26 00:00:00.000	2025-08-25 00:00:00.000	38223	191115	Paid	Lapsed
Harrison Walli	Adya Hans	Life	policy_name	2026-03-20 16:34:12.218650	2023-12-18 00:00:00.000	2026-12-17 00:00:00.000	11976	143712	Paid	Active
Ekapad Hegde	Ojasvi Raju	Life	LIC Jeevan Anand	2026-03-20 16:34:12.218650	2025-02-12 00:00:00.000	2026-02-12 00:00:00.000	29435	588700	Paid	Lapsed
Samar Khanna	Ijaya Wali	Life	Max Life Online Term Plan Plus	2026-03-20 16:34:12.218650	2026-03-02 00:00:00.000	2029-03-01 00:00:00.000	31574	568332	Paid	Active
Theodore Desai	Raagini Nanda	Life	ICIPI Pru Signature	2026-03-20 16:34:12.218650	2024-11-07 00:00:00.000	2025-11-07 00:00:00.000	43023	688368	Paid	Lapsed
Amara Bandi	Amara Deep	Health	policy_name	2026-03-20 16:34:12.218650	2024-12-29 00:00:00.000	2027-12-29 00:00:00.000	32686	326860	Paid	Active
Anjali Chandran	Nirja Bora	Life	ICIPI Protect Smart	2026-03-20 16:34:12.218650	2023-10-24 00:00:00.000	2024-10-23 00:00:00.000	22596	361536	Paid	Lapsed
Ekjaya Kaul	Balveer Chandra	Health	Care Health Joy Plan	2026-03-20 16:34:12.218650	2025-09-18 00:00:00.000	2026-09-18 00:00:00.000	11575	57875	Paid	Active
Hemal Kakar	Nirja Dhar	Life	LIC Jeevan Anand	2026-03-20 16:34:12.218650	2025-01-17 00:00:00.000	2028-01-17 00:00:00.000	65050	1235950	Paid	Active
Adweta Vala	Bachittar Kapur	Life	policy_name	2026-03-20 16:34:12.218650	2024-03-07 00:00:00.000	2026-03-07 00:00:00.000	26180	314160	Paid	Lapsed
Adya Verma	Dhruv Madan	Life	LIC Jeevan Labh	2026-03-20 16:34:12.218650	2025-12-17 00:00:00.000	2028-12-16 00:00:00.000	75410	829510	Paid	Active
Jagdish Trivedi	Rajeshni Pradhan	Health	policy_name	2026-03-20 16:34:12.218650	2026-03-03 00:00:00.000	2029-03-02 00:00:00.000	13778	137780	Paid	Active
Firin Oza	Rachita Basu	Health	Care Health Joy Plan	2026-03-20 16:34:12.218650	2025-02-01 00:00:00.000	2026-02-01 00:00:00.000	76751	690759	Paid	Lapsed

Fig. 18 Dashboard



Menu

- Dashboard
- Customer List
- Sales Agent
- Insurance Companies
- Churn View
- Leads
- Profile

Customers

List of Customers

AddGroups

My Users

Member Name	Email	Phone Number
Nidra Bhakta	senhredhaan@example.com	9856016451
Vedhika Borde	santhenry@example.com	7018271528
Mugdha Viswanathan	tamannasha@example.org	9133648982
Pavani Wason	jacksondutta@example.net	9063611594
Vritti Chandran	dorakritika@example.org	7957774117
Netra Tiwari	zkhosla@example.net	6027011713
Fitan Sodhi	lmitter@example.org	9968491824
Warjas Sahni	sudikshalad@example.com	6612863951
Chaitanya Biswas	mallickradhika@example.org	7598153649
Qadim Prabhakar	vgulati@example.com	9772005325
Nihal Sarin	aboase@example.org	9021975988

Fig. 19 Customers list



Menu

- Dashboard
- Customer List
- Sales Agent
- Insurance Companies
- Churn View
- Leads
- Profile

Sales Agents

List of Sales Agent

AddGroups

Name	Email	Phone Number
Krishna Chacko	tanveer34@example.com	7285961942
Harrison Master	anmolbora@example.net	6997132871
Tristan Mukhopadhyay	dhritiguha@example.net	9144884616
Jagrati Mittal	sekhonhiral@example.org	9955936730
Champak Ravi	baghyawati97@example.com	9733057541
Bhavika Rajan	gautamsinghal@example.com	6921813927
Mahika Chhabra	alexanderlanka@example.org	8755181601
Anjali Kalita	patelchandani@example.com	6345011675
Elijah Dayal	badamiyoshita@example.com	9050686674
Neelima Kakar	isaiah85@example.net	9653672453
Lopa Dutt	chakrabortydayamai@example.net	6563888850
Bakhshi Divan	patlaidika@example.net	9748027547

Fig. 20 Sales Agents List

Insurance Companies

[Add](#)[Back](#)

List

Company Name	Short Name	Website
LIC India	LIC	https://www.khurana.com/
HDFC Life	HDFC	https://krish.net/
ICICI Prudential	ICICI	https://www.seshadri.com/
SBI Life	SBI	https://www.warrior.com/
Max Life	MAX	http://kala.biz/
Star Health	STAR	https://manda.com/
Niva Bupa	BUPA	http://www.dhawan.info/
Care Health	CARE	http://www.patel-ganguly.com/

Fig. 20 Insurance Companies



Menu

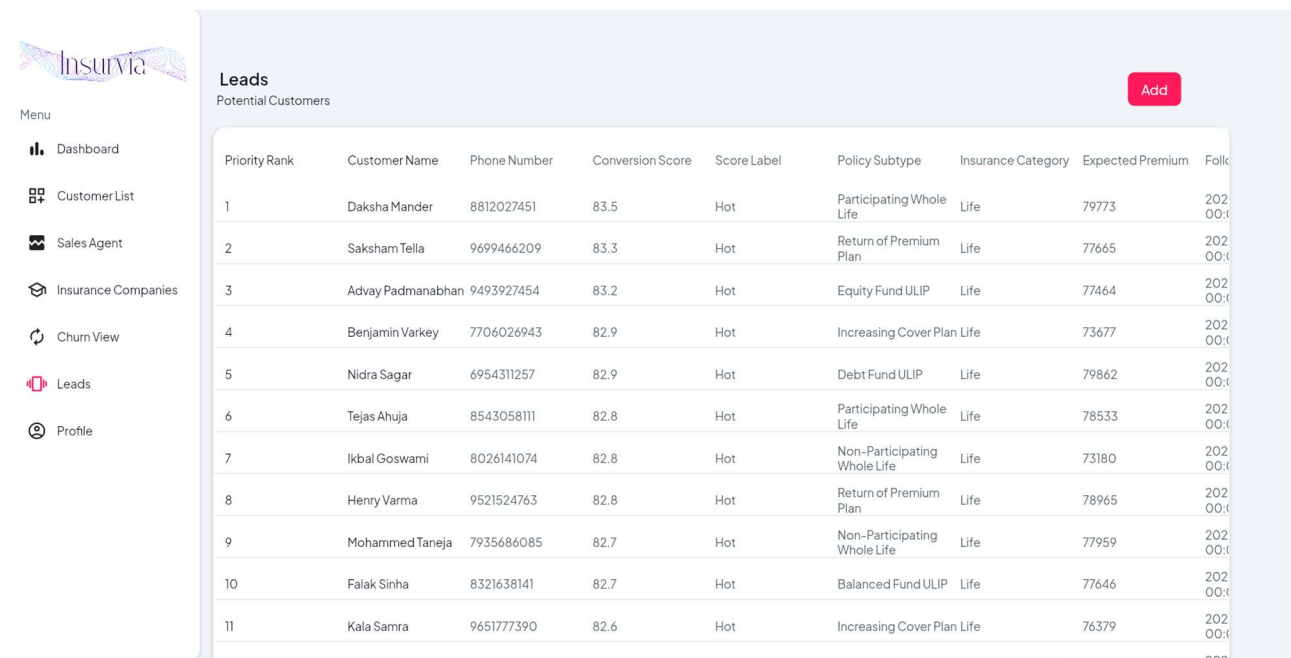
- Dashboard
- Customer List
- Sales Agent
- Insurance Companies
- Churn View
- Leads
- Profile

Churn

Identify At-Risk Customers

Customer Name	Policy Number	Policy Type	Expiry Date	Days to Expiry	Risk Score	Risk Tier
Bina Nagy	POL10938	Life	2026-04-15 00:00:00.000	-2	0.8748	HIGH
Rushil Prasad	POL12903	Health	2026-04-14 00:00:00.000	-3	0.8655	HIGH
Ekalinga Jaggi	POL17313	Life	2026-04-15 00:00:00.000	-2	0.8537	HIGH
Lajita Chana	POL35227	Health	2026-04-14 00:00:00.000	-3	0.8284	HIGH
Gabriel Narang	POL95786	Health	2026-04-13 00:00:00.000	-4	0.8246	HIGH
Bahadurjit Bajaj	POL82828	Health	2026-04-15 00:00:00.000	-2	0.8129	HIGH
Tristan Borah	POL07203	Life	2026-04-11 00:00:00.000	-6	0.7934	HIGH
Amruta Brahmhatt	POL10932	Health	2026-04-15 00:00:00.000	-2	0.7729	HIGH
Kavya Wable	POL49176	Life	2026-04-11 00:00:00.000	-6	0.76	HIGH

Fig. 21 Churn View



Priority Rank	Customer Name	Phone Number	Conversion Score	Score Label	Policy Subtype	Insurance Category	Expected Premium	Follow-up Date
1	Daksha Mander	8812027451	83.5	Hot	Participating Whole Life	Life	79773	2020-01-01
2	Saksham Tella	9699466209	83.3	Hot	Return of Premium Plan	Life	77665	2020-01-01
3	Advay Padmanabhan	9493927454	83.2	Hot	Equity Fund ULIP	Life	77464	2020-01-01
4	Benjamin Varkey	7706026943	82.9	Hot	Increasing Cover Plan Life	Life	73677	2020-01-01
5	Nidra Sagar	6954311257	82.9	Hot	Debt Fund ULIP	Life	79862	2020-01-01
6	Tejas Ahuja	8543058111	82.8	Hot	Participating Whole Life	Life	78533	2020-01-01
7	Iktbal Goswami	8026141074	82.8	Hot	Non-Participating Whole Life	Life	73180	2020-01-01
8	Henry Varma	9521524763	82.8	Hot	Return of Premium Plan	Life	78965	2020-01-01
9	Mohammed Taneja	7935686085	82.7	Hot	Non-Participating Whole Life	Life	77959	2020-01-01
10	Falak Sinha	8321638141	82.7	Hot	Balanced Fund ULIP	Life	77646	2020-01-01
11	Kala Samra	9651777390	82.6	Hot	Increasing Cover Plan Life	Life	76379	2020-01-01

Fig. 22 Lead View

CONCLUSIONS AND FUTURE WORK

The proposed project, Leads and Report Analytics using Machine Learning on Insurance Sales Data, successfully demonstrates the integration of traditional insurance broking operations with modern data-driven technologies. The system provides a centralized platform for managing customers, leads, policies, payments, and commissions, thereby improving efficiency and reducing dependency on manual processes.

A key highlight of the project is the implementation of machine learning models for churn prediction and lead conversion scoring. These models enable the system to generate predictive insights, allowing brokers to identify at-risk customers and prioritize high-potential leads. This not only enhances customer retention strategies but also improves sales performance and decision-making.

The use of FlutterFlow for frontend development and Supabase for backend and database management ensures a scalable and user-friendly application. The interactive dashboard further supports real-time visualization of insights, making the system practical and accessible for end users.

Overall, the project transforms conventional insurance broking into a smart, intelligent, and data-driven system. It highlights the importance of combining database management with machine learning to solve real-world business problems. The system lays a strong foundation for future

enhancements, such as advanced analytics and automation, making it highly relevant in the evolving insurance industry.

The proposed system provides a strong foundation for integrating insurance broking operations with machine learning; however, there are several opportunities for further enhancement and expansion. Future work can focus on improving both the analytical capabilities and overall system functionality.

One potential enhancement is the integration of real-time machine learning models, where predictions are continuously updated based on live data instead of relying on pre-trained models. This would improve the accuracy and responsiveness of churn prediction and lead scoring.

The system can also be extended to include claims management, allowing users to track claim requests, approval status, and settlement details. This would make the platform more comprehensive and closer to real-world insurance systems.

Another area of improvement is the implementation of advanced analytics and visualization, such as trend analysis, forecasting, and interactive charts. These features would provide deeper insights into business performance and support strategic decision-making.

Additionally, the application can be enhanced by developing a mobile version, making it more accessible for agents and brokers who work in the field. Integration with external APIs, such as insurance company systems or payment gateways, can further improve automation and data synchronization.

Security can also be strengthened by implementing advanced authentication mechanisms and role-based access controls. Furthermore, the system can be scaled to support multiple broker organizations, enabling multi-tenant architecture.

Overall, future enhancements aim to make the system more intelligent, scalable, and aligned with real-world industry requirements, ensuring its long-term usability and impact.

8.2 APPLICATIONS

The proposed system has wide practical applications in the insurance broking industry, where efficient customer management, lead tracking, and policy monitoring are essential. It can be used by insurance brokers and agencies to maintain centralized records of customers, leads, active policies, renewals, payments, and commissions in a structured digital environment.

The machine learning features of the system provide valuable applications in business decision-making. The churn prediction model helps organizations identify customers who are likely to discontinue or not renew their policies, enabling timely follow-up and retention strategies.

Similarly, the lead conversion scoring model assists sales teams in prioritizing high-potential leads, thereby improving conversion rates and sales productivity.

The system can also be applied in branch offices or multi-agent insurance firms where multiple users need controlled access to shared data. The dashboard and reporting features support management in tracking performance, monitoring renewals, analyzing sales trends, and evaluating agent productivity.

In future, the same platform can be extended for use in health insurance, life insurance, motor insurance, and corporate insurance management. It can also be adapted by financial service companies that require customer analytics, lead management, and predictive decision support systems.

REFERENCES (APA FORMAT)

- Brownlee, J. (2020). Machine learning algorithms from scratch. Machine Learning Mastery.
- Géron, A. (2019). Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). An introduction to statistical learning (2nd ed.). Springer.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Han, J., Kamber, M., & Pei, J. (2012). Data mining: Concepts and techniques (3rd ed.). Morgan Kaufmann.
- Aggarwal, C. C. (2015). Data mining: The textbook. Springer.
- PostgreSQL Global Development Group. (2024). PostgreSQL documentation. <https://www.postgresql.org/docs/>
- Supabase Inc. (2024). Supabase documentation. <https://supabase.com/docs>
- FlutterFlow Inc. (2024). FlutterFlow documentation. <https://flutterflow.io/>
- IBM Corporation. (2023). What is churn prediction? <https://www.ibm.com/topics/churn-prediction>
- Insurance Regulatory and Development Authority of India (IRDAI). (2024). Insurance industry reports. <https://www.irdai.gov.in/>
- Pressman, R. S. (2014). Software engineering: A practitioner's approach (8th ed.). McGraw-Hill.
- Sommerville, I. (2016). Software engineering (10th ed.). Pearson.
- Kotler, P., & Keller, K. L. (2016). Marketing management (15th ed.). Pearson.

APPENDIX

1. Plagiarism and AI report

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **104 Not Cited or Quoted 13%**
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 8%  Internet sources
- 5%  Publications
- 9%  Submitted works (Student Papers)

2. Project offer letter



+91 9923052627
SR.NO.274/1 & OTHERS ,SKY-I SONGBIRDS, TOWER NO7,FLAT NO 607,AT
POST BHUGAON MULSHI Pune Maharashtra 412115 INDIA
kapil@simplesight.co.in
www.simplesight.co.in

Project Sponsorship Allocation letter

To,
The Head,
CSE AIML, VIIT
Pune

We confirm that the project "Leads and report analytics using machine learning on insurance sales data" is allocated to the following team

Sr. No.	Roll no.	PRN	Name of the students	Email ID
1.	391056	22311783	Gauri Mane	gauri.22311783@viit.ac.in
2.	391057	22311784	Sarthak Hambir	sarthak.22311784@viit.ac.in
3.	391061	22311792	Raghav Bhise	raghav.22311792@viit.ac.in
4.	391064	22311799	Joel Cherian	cherian.22311799@viit.ac.in

We hope that your team will sincerely complete it in the duration from 06/01/26 to 05/04/26 in the **SimpleSight Solutions Pvt. Ltd.** under the guidance of the **Industrial Mentor, Mr. Kapil Khot** and **Faculty Mentor, Dr. Gitanjali Shinde.**

Students will receive access to technical mentorship, design review sessions, and prototype evaluation assistance during the project. We welcome the team to this academic collaboration industry and look forward to impactful outcomes from this engagement.

Kapil Khot

A handwritten signature in black ink, appearing to be "K. Khot", written over a light blue circular stamp.



Director at SimpleSight Solutions Pvt. Ltd.

3. Project completion letter



+91 9923052627
SR.NO.274/1 & OTHERS ,SKY-I SONGBIRDS, TOWER NO7,FLAT NO 607,AT
POST BHUGAON MULSHI Pune Maharashtra 412115 INDIA
kapil@simplesight.co.in
www.simplesight.co.in

Project Completion Certificate

To,
The Head,
CSE AIML, VIIT
Pune

We **SimpleSight Solutions Pvt Ltd.** hereby certify that the following students of Third Year BTech CSE_AIML, VIIT have completed their Third year BTech project titled: “**Leads and report analytics using machine learning on insurance sales data**” at Vishwakarma Institute of Information Technology, Kondhwa in the Academic Year: 2025-26 under the guidance of the **Industrial Mentor, Mr. Kapil Khot** and **Faculty Mentor, Dr. Gitanjali Shinde**

Sr. No.	Roll no.	PRN	Name of the students	Email ID
1.	391056	22311783	Gauri Mane	gauri.22311783@viit.ac.in
2.	391057	22311784	Sarthak Hambir	sarthak.22311784@viit.ac.in
3.	391061	22311792	Raghav Bhise	raghav.22311792@viit.ac.in
4.	391064	22311799	Joel Cherian	cherian.22311799@viit.ac.in

We thank them for their dedication and technical contribution and wish them good luck for the future.

A handwritten signature in black ink, appearing to be "K. Khot", written over a light blue horizontal line.

Kapil Khot
Director at SimpleSight Solutions Pvt. Ltd.

